INSTITUTO UNIVERSITÁRIO DE LISBOA

Unidade Curricular: Arquitetura e Desenho de Software

Relatório do Projeto – Ano Letivo 2025/2026 | 1º Semestre



2025-2026-ADS-Grupo_Diurno-A

Elementos do Grupo:

Ema Silva Reis, Nº 110847

Mariana de Almeida Capela, Nº 104715

Nádia Patrícia Gavancha, Nº 110805

Mestrado em Engenharia Informática

Dezembro de 2025

Índice

1. Introdução 1
2. Enquadramento Teórico 1
2.1. Métricas de Qualidade de Software 2
2.2. Análise Estática e Árvore de Sintaxe Abstrata (AST) 5
3. Requisitos do Sistema 5
4. Metodologia de Desenvolvimento 8
4.1. Organização da Equipa 8
4.2. Estrutura do Projeto Scrum 9
4.3. Product Backlog 9
4.4. Sprints 10
5. Desenho e Arquitetura do Software 13
5.1. Decisões tecnológicas 14
5.2. Modelação UML 15
5.3. Arquitetura do Software 18
6. Implementação 19
6.1. Ambiente de Desenvolvimento 20
6.2. Integração da API JDT AST 20
6.3. Recolha de Métricas e Processamento 20
6.4. Paralelismo entre a Implementação e a Arquitetura 21
7. Testes e Validação 22
7.1. Testes unitários 23
7.2. Testes dos atributos de qualidade 24
8. Resultados e Avaliação 27
8.1. Enquadramento do Processo 27
8.2. Execução da ferramenta 28
8.3. Avaliação da Ferramenta 30
9. Conclusões e Trabalho Futuro 35
Bibliografia 37

Índice de Imagens

Figura 1. Utility Tree 6
Figura 2. Product Backlog no Trello 10
Figura 3. Exemplo Sprint Planning do Sprint 2 11
Figura 4. Exemplo de tarefa do Sprint Backlog 11
Figura 5. Estrutura do daily scrum no Trello 11
Figura 6. Respostas às questões do daily Scrum 11
Figura 7. Excerto do Sprint Review (Sprint 2) 12
Figura 8. Sprint Retrospective (Sprint 2) 12
Figura 9. Burndown Chart Sprint 3 12
Figura 10. Diagrama de Casos de Uso 15
Figura 11. Diagrama de Classes Conceptual 16
Figura 12. Diagrama de Implementação 17
Figura 13. Esquema da arquitetura do software da ferramenta 18
Figura 14. Diagrama BPMN 27
Figura 15. Página inicial da ferramenta 28
Figura 16. Erro apresentado ao tentar analisar e nenhum ficheiro foi selecionado 28
Figura 17. Erro apresentado ao tentar analisar um projeto/pasta do tipo incorreto 28
Figura 18. Resultados para a análise 29
Figura 19. Visualização gráfica da métrica NOM por classe 29
Figura 20. Pasta selecionada para a criação do CSV 30
Figura 21. Alerta de sucesso na criação do ficheiro 30
Figura 22. Alerta de erro 30
Figura 23. Pasta com o ficheiro criado 30
Figura 24. Conteúdo do ficheiro criado 30
Figura 25. Excerto dos resultados para as métricas da Classe AtributeInfo 32
Figura 26. Parte dos resultados da ferramenta CodeMR 34
Figura 27. Resto dos resultados da ferramenta CodeMR 34

Índice de Tabelas

Tabela 1. Cenários de Atributos de Qualidade e Priorização 7
Tabela 2. Sprints executados 12
Tabela 3. Resultados das métricas para o Projeto 31
Tabela 4. Resultados das métricas para o Projeto com MetricsTree 32
Tabela 5. Resultados das métricas para o Projeto com CodeMR 34

1. Introdução

A qualidade do código é um fator essencial no desenvolvimento de software, influenciando diretamente a manutenção, a *performance* e a evolução de um projeto. No entanto, analisar manualmente a estrutura e a complexidade de um sistema pode ser um processo demorado e propenso a erros, especialmente em projetos de maior dimensão. Surge assim a motivação para desenvolver uma ferramenta que automatize a análise de métricas de código, permitindo avaliar a qualidade do software de forma mais rápida e eficiente.

Este projeto tem como objetivo principal criar uma ferramenta que permita analisar automaticamente projetos Java e calcular métricas de qualidade a partir do código fornecido. A aplicação baseia-se na **API Eclipse JDT** e no metamodelo **AST** (*Abstract Syntax Tree*) para identificar classes, métodos e atributos, e calcular métricas como *Lines of Code* (LOC), Complexidade Ciclomática, entre outras. Além disso, a ferramenta deve possuir uma interface gráfica simples que apresente os resultados e permita exportar os mesmos para um ficheiro CSV.

Este relatório encontra-se estruturado da seguinte forma:

- Capítulo 2 – apresenta o enquadramento teórico, abordando as métricas de qualidade e AST;
- Capítulo 3 – detalha a análise e especificação do sistema, incluindo os requisitos funcionais, não funcionais e opcionais;
- Capítulo 4 – descreve a metodologia de desenvolvimento adotada e a forma como o projeto foi gerido com base no *Scrum*;
- Capítulo 5 – aborda o desenho e a arquitetura do software, apresentando as principais decisões técnicas e os diagramas UML;
- Capítulo 6 – apresenta a implementação do sistema;
- Capítulo 7 – descreve os testes e validação realizados;
- Capítulo 8 – apresenta os resultados obtidos e a respetiva avaliação;
- Capítulo 9 – inclui as conclusões e sugestões para trabalho futuro.

2. Enquadramento Teórico

A Qualidade do Código é um fator determinante para a sustentabilidade e sucesso de um sistema de software, influenciando diretamente a sua facilidade de manutenção, testabilidade e capacidade de evolução. Contudo, avaliar qualidade de forma exclusivamente subjetiva tende a ser insuficiente, sobretudo em projetos que crescem em dimensão e complexidade.

Assim, torna-se essencial recorrer a mecanismos objetivos que permitam caracterizar o estado do código, identificar padrões estruturais e sinalizar potenciais áreas de risco. É neste contexto que as Métricas de Qualidade de Software se tornam essenciais.

2.1. Métricas de Qualidade de Software

As métricas de software são então medidas quantitativas que avaliam propriedades e características do código, fornecendo uma base objetiva para a avaliação e monitorização de um produto de software.

Isoladamente, cada métrica oferece apenas uma perspetiva parcial, as conclusões mais sólidas e fiáveis emergem da combinação de várias métricas e da sua evolução ao longo do tempo.

O cálculo destas métricas permite benefícios práticos num ciclo de desenvolvimento, por exemplo:

- **Deteção precoce de risco:** Identificação de *code smells* e anomalias (classes/métodos muito complexos ou com grande acoplamento), permitindo intervenção antecipada;
- **Priorização de esforço:** Orientação do *refactoring* e dos esforços de teste para áreas de maior impacto (como componentes críticos ou com menos qualidade);
- **Monitorização e Melhoria contínua:** Fornecimento de *insights* para monitorizar tendências de qualidade ao longo do tempo, suportando um processo de desenvolvimento iterativo e de melhoria contínua.

A nossa ferramenta de cálculo de métricas analisa projetos *Java*. As métricas consideradas para a avaliação de cada projeto são:

LOC (Lines of Code)

Esta é uma métrica de dimensão (tamanho) do código, útil para contextualizar esforço e comparar módulos, contando a quantidade de linhas de código de um programa.

LOC pode ser calculado de diferentes maneiras, podendo contar com todas as linhas incluindo linhas em branco ou comentários (*physical LOC*), ou pode contar apenas *statements* executáveis e declarações (*logical LOC*).

Apesar desta métrica conseguir fornecer uma medida básica do tamanho de um projeto/classe, é uma métrica limitada, na medida em que não consegue refletir com precisão a complexidade ou qualidade do código.

NOM (Number of Methods)

Esta métrica conta o número total de métodos definidos numa classe. Pode incluir todos os tipos de métodos, como construtores, métodos públicos, privados e protegidos, dependendo da abordagem de contagem adotada.

Embora forneça uma indicação básica sobre a quantidade de operações realizadas por uma classe, é uma métrica limitada, pois não avalia diretamente a qualidade, coesão ou acoplamento do código. No entanto, valores elevados podem indicar uma maior complexidade da classe, tornando-a potencialmente mais difícil de compreender, testar e manter.

NOA (Number of Attributes)

A métrica NOA (*Number of Attributes*) mede o número total de atributos declarados numa classe. Estes atributos representam o estado interno do objeto e podem incluir variáveis públicas, privadas, protegidas ou estáticas, consoante o critério de contagem utilizado.

Apesar de fornecer uma visão geral sobre o tamanho e a complexidade estrutural de uma classe, esta métrica é limitada, uma vez que não reflete diretamente a forma como os atributos são utilizados ou relacionados entre si. Ainda assim, um valor elevado de NOA pode indicar uma classe excessivamente complexa ou com demasiadas responsabilidades sobre os seus dados.

MCC (McCabe's Cyclomatic Complexity)

Esta métrica, criada e apresentada por Thomas McCabe, representa a complexidade de um programa, verificando o número de decisões que o programa tem de fazer. Este valor é calculado através da fórmula:

$$C = D + 1$$

Considerando que o programa possui apenas um ponto de entrada e saída, a fórmula acima é utilizada para calcular o MCC de um método, onde D representa o número de decisões feitas pelo código (*if*, *while*, *for*, entre outros). Para calcular o MCC de uma classe, soma-se as complexidades dos vários métodos da classe:

$$C_{classe} = \sum_{i=0}^{n} C_n$$

Quanto menor a complexidade do código, mais fácil é compreendê-lo e, por sua vez, modificá-lo e testá-lo. Por outro lado, códigos com maior complexidade apresentam uma maior probabilidade de ocorrência de *bugs*. Dessa forma, valores entre 1 e 10 são considerados aceitáveis; de 11 a 20, representam códigos complexos; de 21 a 50, indicam alta complexidade; e acima de 50, são classificados como extremamente complexos, exigindo revisão imediata.

CBO (Coupling Between Objects)

Esta métrica calcula o número de classes diferentes às quais uma classe está "acoplada", através de chamada a métodos, parâmetros, argumentos de métodos, herança, tipos de retorno ou variáveis em métodos. Ou seja, esta métrica conta quantas outras classes a classe referencia no seu código.

Acoplamento entre classes é necessário e inevitável para o funcionamento de um sistema, no entanto, acoplamento em excesso torna o sistema mais difícil de manter e reutilizar, aumentando custo de mudança e dificuldade de testes.

DIT (Depth of Inheritance Tree)

Esta métrica representa a profundidade de uma classe numa hierarquia de heranças, ou seja, refere-se à distância entre a classe a analisar e a classe raiz da árvore de heranças num sistema de programação orientada a objetos.

Embora a presença de uma árvore de heranças promova a reutilização de código, o excesso de profundidade pode causar problemas (*code smells*). Entre eles estão o aumento da complexidade, o que torna mais difícil a compreensão do código, o aumento da dificuldade

em realizar manutenção devido ao número elevado de métodos e atributos herdados e o aumento de risco de erros, pois modificações em uma classe podem causar problemas nas várias classes que a herdam.

Desta forma, o valor de DIT recomendado é de 5 ou menos, sendo que todas as classes por *default* herdam a classe *Object*, ou seja, no mínimo, DIT é igual a 1.

NOC (Number of Children)

A métrica NOC (*Number of Children*) representa o número de subclasses que herdam diretamente de uma determinada classe. É uma métrica de herança que permite avaliar o grau de reutilização e de generalização do código num sistema orientado a objetos.

Um valor elevado de NOC pode indicar que a classe pai é amplamente reutilizada, o que tende a ser positivo em termos de extensibilidade e abstração. No entanto, também pode sugerir uma dependência elevada entre as subclasses e a superclasse, tornando mais arriscadas eventuais alterações na classe base, já que estas podem propagar efeitos indesejados às classes derivadas.

Por outro lado, um valor baixo de NOC pode indicar baixo potencial de reutilização ou uma hierarquia pouco explorada, embora nem sempre isso seja negativo.

Assim, o NOC fornece uma visão equilibrada entre reutilização e complexidade, ajudando a identificar classes centrais na hierarquia de herança que merecem especial atenção no processo de manutenção e evolução do software.

NPRM (Number of Parameters per Method)

Esta métrica calcula a media de parâmetros que cada método da classe a analisar recebe. Para fazer este cálculo utilizamos a fórmula:

$$NPRM = \frac{Número\ total\ de\ parâmetros\ dos\ vários\ métodos}{número\ de\ métodos}$$

Desta forma, para calcular este valor, verifica-se quantos parâmetros são recebidos no total, ou seja, é somado o número de parâmetros que cada método recebe, e depois dividimos esse valor pelo número total de métodos da classe.

Um NPRM alto indica-nos que a classe tem demasiadas responsabilidades, o que a torna mais difícil de compreender, testar e fazer manutenção. Para além disso, pode representar um acoplamento mais alto do que o recomendado.

AvgLOCMethod (Average LOC per Method)

Esta métrica mede o "peso médio" nos métodos de uma classe, ou seja, quantas linhas de código tem, em média, cada método. Calculando-se da seguinte maneira:

$$AvgLOCMethod = \frac{Número\ total\ de\ linhas\ dos\ vários\ métodos}{Número\ de\ métodos\ da\ classe}$$

Este valor permite analisar a tendência do tamanho dos métodos e apoiar decisões de *refactoring*. Quando a média é alta, isto indica-nos que os métodos estão tendencialmente longos, havendo risco de baixa coesão e de múltiplas responsabilidades.

2.2. Análise Estática e Árvore de Sintaxe Abstrata (AST)

No âmbito deste projeto, a obtenção das métricas é suportada por análise estática. Análise estática consiste em a analisar o código-fonte sem executar o programa. Este tipo de análise permite caracterizar propriedades estruturais do software de forma previsível e repetível, sendo particularmente adequado para automatizar a extração de indicadores de qualidade.

Para tornar a análise mais fiável e menos dependente de aspetos como formatação, espaçamento ou comentários, a ferramenta recorre à construção de uma Árvore de Sintaxe Abstrata (AST, *Abstract Syntax Tree*).

A AST representa o código como uma estrutura hierárquica, cada nó corresponde a uma construção sintática relevante, como declarações de classes, métodos, atributos e estruturas de controlo. Esta representação permite identificar e contar estes elementos de forma sistemática e consistente, evitando limitações típicas de abordagens baseadas apenas em pesquisa textual.

Deste modo, a AST constitui a base do processo de extração de informação, a partir dela serão extraídos os elementos necessários ao cálculo das métricas, permitindo apresentar resultados por classe e suportar a interpretação da qualidade do código analisado.

3. Requisitos do Sistema

Como referido anteriormente, a ferramenta proposta, *Ferramenta de Análise de Métricas de Código*, tem como objetivo automatizar o processo de obtenção de métricas de qualidade de software a partir do código fonte de projetos desenvolvidos em Java.

Os requisitos devem cobrir todo o ciclo de utilização da aplicação, desde a seleção do projeto a analisar até à obtenção dos resultados e a sua possível exportação.

Os principais requisitos são então:

- **Seleção de projeto:** o sistema deve permitir que o utilizador selecione uma pasta/projeto *Java* para análise.
- **Leitura automática de ficheiros**: o sistema deve identificar automaticamente os ficheiros *.java* do projeto selecionado.
- **Geração da AST:** através da utilização da *API Eclipse JDT* a aplicação deve construir a AST de cada ficheiro.
- **Identificação de classes e métodos:** a aplicação deve conseguir extrair a lista de classes, métodos e atributos encontrados.
- **Cálculo das métricas de qualidade:** a aplicação deve conseguir calcular as seguintes métricas:
    - LOC (Lines of Code)
    - NOM (Number of Methods)
    - NOA (Number of Attributes)
    - MCC (McCabe's Cyclomatic Complexity)
    - CBO (Coupling Between Objects)
    - DIT (Depth of Inheritance Tree)
    - NOC (Number of Children)

  - NPRM (Number of Parameters per Method)
  - AvgLOCMethod (Average LOC per Method)
- **Apresentação dos resultados:** o sistema deve apresentar visualmente os resultados obtidos, numa tabela e/ou gráficos.
- **Exportação dos resultados:** o sistema deve permitir exportar os resultados para um ficheiro CSV.
- **Mensagens de erro e validação:** em caso de erros ou problemas na análise do código devem ser apresentadas mensagens claras que identifiquem o problema.

Atributos de qualidade

Para além das funcionalidades descritas, o sistema deve ainda assegurar um conjunto de atributos de qualidade que garantam que o processo de análise decorre de forma consistente, com desempenho adequado e boa usabilidade ao longo do processo de análise. A ***Utility Tree*** apresentada abaixo deriva estes objetivos de qualidade em cenários concretos.

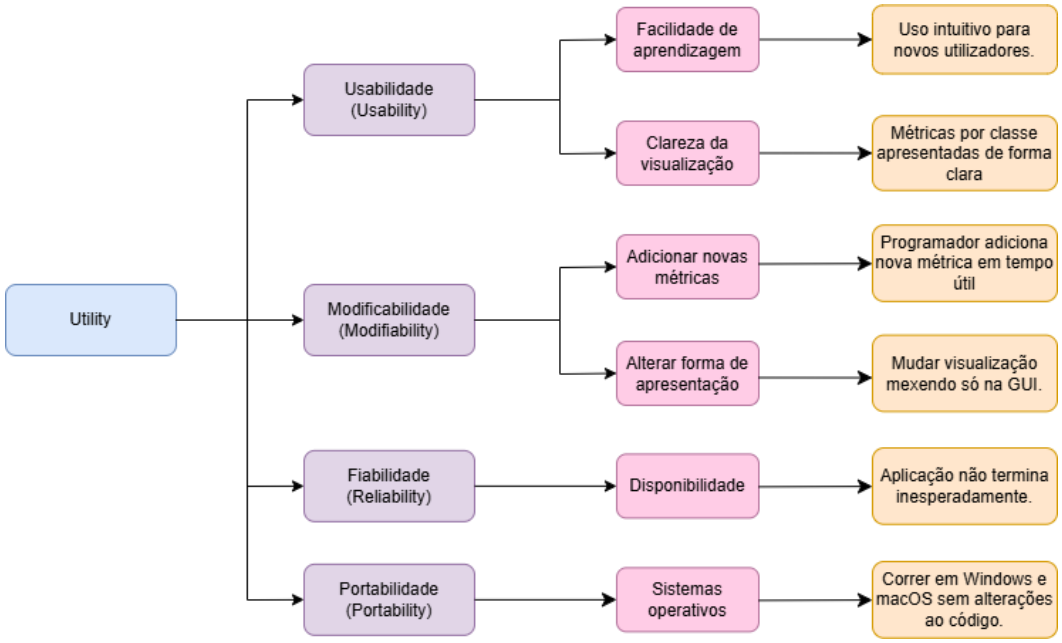


Figura 1. Utility Tree

A árvore destaca os atributos de qualidade de alto nível considerados críticos para o projeto. Para os tornar operacionais e verificáveis, estes atributos foram decompostos em cenários de qualidade específicos.

A tabela abaixo detalha cada folha da árvore, associando-lhe uma prioridade com base no valor para o negócio e no impacto arquitetónico, permitindo direcionar o esforço de desenvolvimento para as áreas mais críticas.

A coluna **"Prioridade"** utiliza a notação **(Valor, Impacto)**, onde o primeiro elemento corresponde ao Valor de Negócio e o segundo ao Impacto Arquitetónico. A escala utilizada é **H** (*High*/Alto), **M** (*Medium*/Médio) e **L** (*Low*/Baixo), com os seguintes significados:

1. **Valor de Negócio:**
   - **H (Alto):** Requisito essencial;
   - **M (Médio):** Requisito importante, mas cuja ausência não compromete o projeto;
   - **L (Baixo):** Requisito desejável, mas que não justifica um elevado esforço de desenvolvimento.
2. **Impacto Arquitetónico:**
   - **H (Alto):** A satisfação deste requisito afeta profundamente a arquitetura;
   - **M (Médio):** Afetará parcialmente a arquitetura;
   - **L (Baixo):** Tem pouco impacto na estrutura base do software.

Tabela 1. Cenários de Atributos de Qualidade e Priorização

Atributo de Qualidade	**Refinamento**	**Cenário (Descrição)**	**Prioridade**
Usabilidade	Facilidade de Aprendizagem	Um novo utilizador deve conseguir utilizar a ferramenta intuitivamente sem necessidade de ler documentação extensa.	(H, L)
	Clareza da Visualização	As métricas (LOC, CBO, etc.) devem ser apresentadas por classe numa tabela clara.	(H, M)
Modificabilidade	Extensibilidade	Um programador deve conseguir adicionar uma nova métrica em tempo útil, aproveitando a estrutura da AST.	(M, H)
	Flexibilidade da UI	Deve ser possível alterar a forma de visualização dos dados (ex: gráficos) alterando apenas a camada da GUI, sem afetar a lógica de cálculo.	(L, L)
Fiabilidade	Disponibilidade	A aplicação deve manter-se estável e não terminar inesperadamente durante a análise de projetos grandes.	(H, M)
Portabilidade	Adaptabilidade de SO	A aplicação deve correr em ambientes *Windows* e *macOS* sem qualquer	(H, L)

Atributo de Qualidade	**Refinamento**	**Cenário (Descrição)**	**Prioridade**
		alteração ao código fonte, tirando partido da JVM.	

Da priorização apresentada, é possível identificar os atributos de qualidade que exigem maior atenção no desenho da arquitetura.

Destacam-se sobretudo a *Usabilidade*, a *Fiabilidade* e a *Portabilidade*, todas classificadas com elevado valor para o negócio, refletindo a necessidade de uma interface intuitiva, de uma execução robusta e de compatibilidade entre sistemas operativos. Por sua vez, a *Modificabilidade* apresenta um cenário com impacto arquitetónico elevado, evidenciando a importância de uma estrutura interna que facilite a introdução de novas métricas e a evolução futura da aplicação.

Esta leitura global dos cenários permite compreender quais as qualidades que deverão ser suportadas pelo sistema e servirá de base para a definição dos requisitos não funcionais apresentados na secção seguinte.

4. Metodologia de Desenvolvimento

O sucesso de um projeto de engenharia de software depende não apenas das decisões técnicas, mas também da eficácia dos processos de gestão e organização da equipa. Este capítulo descreve a metodologia de desenvolvimento adotada para a criação da Ferramenta de Análise de Métricas de Código, detalhando a forma como o trabalho foi planeado, executado e monitorizado.

A equipa optou pela utilização do ***Scrum***, um *framework* de metodologia ágil, que permitiu uma abordagem iterativa e incremental ao desenvolvimento. Esta escolha teve como objetivo facilitar a adaptação a eventuais mudanças nos requisitos, promover a entrega contínua de funcionalidades e potenciar a colaboração entre os membros do grupo.

Nas secções seguintes, são apresentados a organização da equipa, a estrutura das cerimónias *Scrum* realizadas, a gestão do *Product Backlog* e a descrição detalhada dos sprints.

4.1. Organização da Equipa

A equipa de desenvolvimento é constituída por três elementos.

Embora a metodologia *Scrum* defina papéis distintos (*Product Owner*, *Scrum Master* e *Development Team*), a natureza académica do projeto e a dimensão reduzida da equipa exigiram uma adaptação flexível destas responsabilidades. Para garantir que todos os membros participavam ativamente na implementação técnica e adquiriam conhecimentos variados, optou-se por uma abordagem colaborativa onde os papéis foram desempenhados da seguinte forma:

- ***Development Team***: Todos os membros da equipa atuaram como developers, participando ativamente na escrita de código, testes e documentação ao longo de todos os sprints.
- ***Product Owner***: A definição e priorização dos requisitos no *Product Backlog* foi realizada em conjunto pela equipa, com base no enunciado do projeto e no *feedback* recebido nas aulas.
- ***Scrum Master***: A gestão do processo, a garantia do cumprimento das cerimónias (como as *Daily Scrums*) e a remoção de impedimentos foram responsabilidades partilhadas, assegurando que o foco da equipa se mantinha nos objetivos do *sprint*.

4.2. Estrutura do Projeto Scrum

Neste projeto *Scrum* foram realizados 5 *sprints*, cada um com 2 semanas, totalizando 10 semanas de desenvolvimento de projeto.

Em cada *sprint* foram realizadas **cerimónias *Scrum***, para ajudar a acompanhar o desenvolvimento da ferramenta. Assim, foram realizados:

- ***Sprint Plannings***, antes de cada *sprint* com o objetivo de escolher quais os objetivos do *Backlog* vão ser realizados;
- ***Daily Scrums***, feitos, no nosso caso, no fim de um dia de trabalho no projeto, para dar a conhecer:
  - *O que foi feito hoje?*
  - *O que será feito amanhã?*
  - *Que obstáculos foram sentidos?*
- ***Sprint Reviews,*** realizados no fim de cada sprint, onde verificamos se os objetivos foram cumpridos e validados;
- ***Sprint Retrospectives***, também realizados no fim dos sprints, onde verificamos o que correu bem, o que correu mal e o que pode ser melhorado no próximo sprint.

Para fazer a gestão de desenvolvimento do projeto utilizámos o **Trello**, uma ferramenta que permite implementar a estrutura *Scrum*, e o **GitHub**, para gerir o projeto e as suas versões ao longo do seu desenvolvimento.

4.3. Product Backlog

O ***Product Backlog*** agregou todas as tarefas necessárias para a conclusão do projeto, desde a fase inicial de gestão até à entrega final. Este artefacto funcionou como uma lista dinâmica, contendo não apenas os requisitos funcionais, mas também tarefas de desenho de arquitetura e atividades de documentação.

Como referido anteriormente, a gestão destes itens foi realizada através da plataforma **Trello**, onde cada grande entrega foi mapeada num cartão.



Figura 2. Product Backlog no Trello

A figura 2 ilustra a organização visual do *Product Backlog*, apresentando o conjunto de tarefas realizadas.

Conforme visível na imagem, as tarefas listadas cobrem todo o âmbito do desenvolvimento:

- **Gestão e Arquitetura**: Tarefas fundamentais de arranque, identificadas nos cartões "Gestão e Planeamento" e "Desenho e Arquitetura do Projeto".
- **Funcionalidades de Código**: Implementação dos requisitos nucleares, nomeadamente a "Leitura Automática do Código", o "Cálculo de Métricas", a "Interface Gráfica" e a "Exportação de Resultados para CSV".
- **Documentação**: Inclusão da escrita do relatório final como uma tarefa explícita ("Documentação – Relatório").

Uma característica importante desta organização é a utilização de etiquetas coloridas (*labels*) visíveis em cada cartão. Estas etiquetas indicam o *Sprint* em que cada tarefa foi realizada. A presença de cartões com múltiplas cores demonstra que essas tarefas foram trabalhadas de forma incremental ou contínua ao longo de vários *sprints*, enquanto outras (como a "Gestão e Planeamento") ficaram concluídas numa fase única.

4.4. Sprints

Com base no *Product Backlog* (descrito acima), as tarefas foram distribuídas ao longo dos *sprints*, permitindo a entrega de incrementos funcionais e demonstráveis da aplicação ao longo do desenvolvimento.

Antes do início de cada *sprint* realizou-se um *Sprint Planning*, na qual foram definidos:

- O(s) objetivo(s) do *sprint* (itens selecionados do *Product Backlog*);
- A decomposição dos itens em tarefas técnicas de implementação, teste e documentação (para integrarem o *Sprint Backlog*);
- A atribuição de responsabilidades por elemento da equipa;
- A estimativa de esforço e a calendarização das tarefas.



Figura 3. Exemplo Sprint Planning do Sprint 2

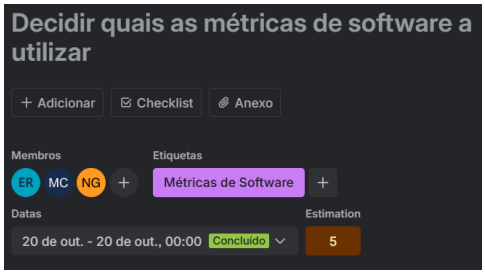


Figura 4. Exemplo de tarefa do Sprint Backlog

Ao longo de cada sprint, foram realizadas ***Daily Scrums*** com o objetivo de acompanhar o progresso das tarefas e identificar impedimentos. No entanto, por se tratar de um projeto universitário de equipa sem dedicação diária contínua, estas reuniões não ocorreram todos os dias, sendo realizadas apenas quando necessário (nomeadamente nos dias de trabalho efetivo), mantendo-se ainda assim a lógica de alinhamento regular sobre: o que foi feito, o que falta fazer e quais os obstáculos existentes.



Figura 5. Estrutura do daily scrum no Trello

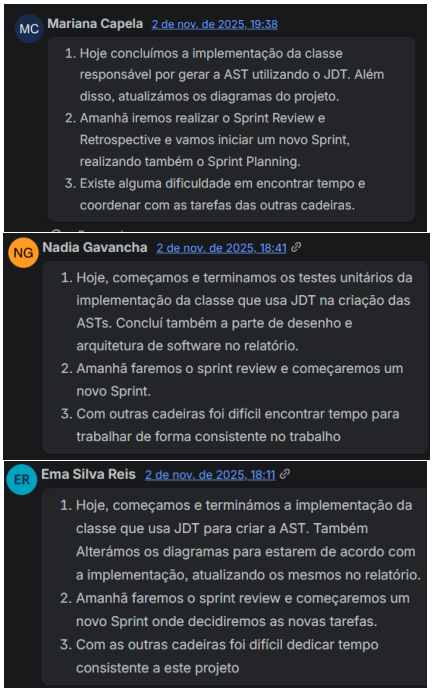


Figura 6. Respostas às questões do daily Scrum

Por fim, no final de cada sprint foram realizadas as cerimónias de fecho:

A ***Sprint Review***, onde foi verificado o incremento desenvolvido e cumprimento dos objetivos estipulados. As funcionalidades implementadas foram validadas, sendo

aprovadas quando cumpriam o esperado, ou rejeitadas e devolvidas ao *backlog* quando incompletas, com erros ou necessidade de revisão.

Na ***Sprint Retrospective***, a equipa refletiu sobre o processo e a colaboração durante o sprint, identificando o que correu bem, o que correu menos bem e ações concretas de melhoria a aplicar no sprint seguinte.

Adicionalmente, foi utilizado um *Burndown Chart* para monitorizar a evolução do trabalho ao longo do sprint, comparando o esforço planeado com o esforço efetivamente concluído.

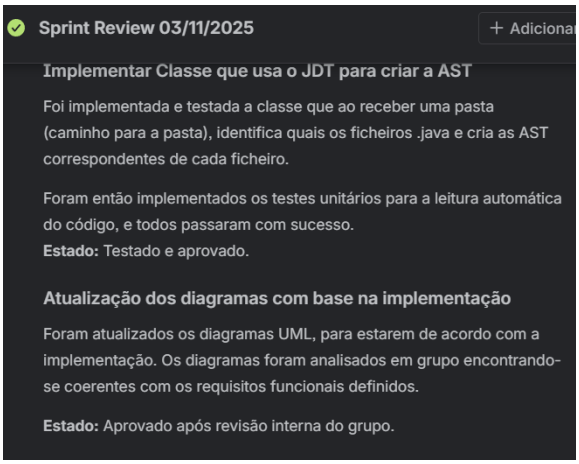


Figura 7. Excerto do Sprint Review (Sprint 2)

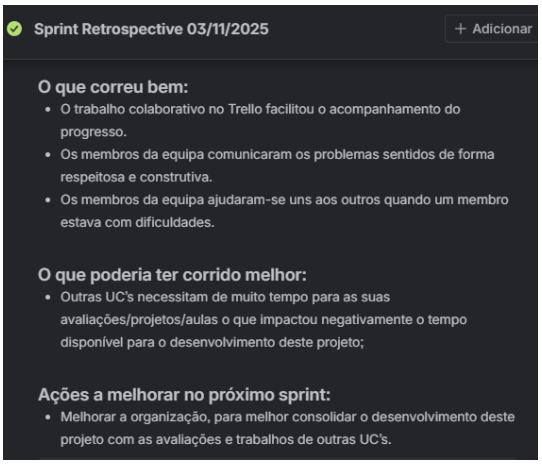


Figura 8. Sprint Retrospective (Sprint 2)

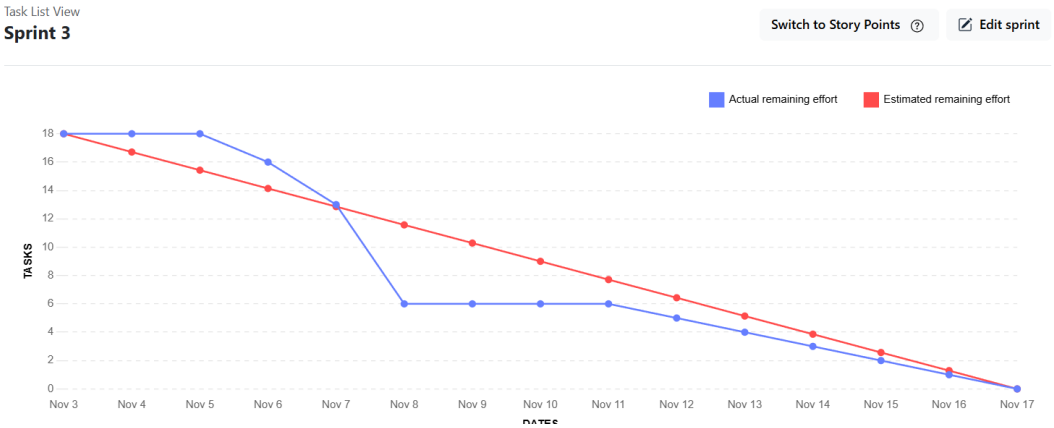


Figura 9. Burndown Chart Sprint 3

De seguida, apresenta-se o detalhe de cada sprint, incluindo os objetivos definidos e os resultados entregues em cada um deles.

Tabela 2. Sprints executados

Sprint	Data início	Data fim	Objetivos	Resultados
Sprint 1	05/10/2025	19/10/2025	- Gestão e planeamento do projeto; - Criação do repositório, estrutura inicial e arranque do relatório;	Foi concluída a criação do repositório, a estrutura inicial do projeto e o arranque do relatório, incluindo os diagramas UML. A leitura automática do

			- Desenho e arquitetura do software (diagramas UML); - Início da leitura automática do código.	código ficou adiada para o sprint seguinte.
Sprint 2	20/10/2025	02/11/2025	- Implementação da leitura automática do código; - Atualização dos diagramas conforme a implementação.	Objetivos foram cumpridos
Sprint 3	03/11/2025	16/11/2025	- Implementação do cálculo de métricas; - Implementação da interface gráfica inicial (sem exportação CSV).	Objetivos foram cumpridos
Sprint 4	17/11/2025	30/11/2025	- Exportação de resultados para CSV; - Atualização/melhoria da GUI.	Objetivos foram cumpridos
Sprint 5	01/12/2025	16/12/2025	- Finalização do relatório.	Objetivos foram cumpridos

Na distribuição do trabalho ao longo dos cinco sprints, verificou-se uma evolução incremental da aplicação, com entregas progressivas desde a preparação e desenho inicial até à implementação completa das funcionalidades e respetiva documentação.

Apesar de no *Sprint* 1 parte do trabalho funcional ter sido adiado, o planeamento foi ajustado nos sprints seguintes, assegurando o cumprimento dos objetivos definidos e a entrega final do projeto dentro do calendário estabelecido.

5. Desenho e Arquitetura do Software

Este capítulo dedica-se à apresentação da **arquitetura e do desenho técnico da solução**. Partindo dos requisitos estabelecidos na fase de análise, detalha-se a estrutura que suporta a aplicação e fundamentam-se as opções tecnológicas que sustentam o funcionamento da *Ferramenta de Análise de Métricas de Código*.

O **desenho de software** é uma etapa fundamental para garantir que a aplicação é robusta, escalável e de fácil manutenção. Nesse sentido, este capítulo encontra-se estruturado em três secções principais:

1. **Decisões Tecnológicas**: Fundamentação das escolhas relativas à linguagem de programação, bibliotecas (como a JDT Core), ferramentas de *build* e controlo de versões;
2. **Modelação UML**: Representação visual do sistema através de diagramas de Casos de Uso, Classes e de Implementação, que ilustram tanto a estrutura estática como o comportamento dinâmico da aplicação;

3. **Arquitetura do Software**: Análise dos padrões arquiteturais adotados, nomeadamente a organização em camadas e o padrão *Model-View-Controller* (MVC), que asseguram a modularidade do sistema.

Esta estruturação permitiu à equipa ter uma visão clara e partilhada do sistema antes de avançar para a fase de implementação propriamente dita.

5.1. Decisões tecnológicas

Na fase inicial de desenho e arquitetura da ferramenta de **Análise de Métricas de Código**, foram tomadas diversas decisões tecnológicas para garantir um desenvolvimento eficaz, escalável e de fácil manutenção.

Estas decisões foram definidas antes do início do desenvolvimento, de forma a assegurar uma base tecnológica consistente e alinhada com os requisitos do projeto, reduzindo incertezas durante a implementação.

Linguagem e Ambiente de Desenvolvimento:

Decidimos desenvolver a aplicação utilizado a linguagem **Java**, por ser a linguagem-alvo dos projetos analisados e por permitir um tratamento direto e consistente de ficheiros *.java*. Esta escolha facilita também a análise sintática e estrutural do código, nomeadamente através da construção e exploração de Árvores de Sintaxe Abstrata (AST).

Bibliotecas e Análise de Código:

Para a análise de código, optou-se pela utilização da biblioteca **JDT Core** (*org.eclipse.jdt.core*), pertencente ao ecossistema Eclipse JDT, e prevista nos requisitos do projeto. Esta biblioteca disponibiliza mecanismos robustos para criação, análise e manipulação de ASTs e permite extrair informação estrutural (classes, métodos, atributos e relações) necessária ao cálculo das métricas definidas.

Exportação dos Resultados

Para a exportação de resultados, decidiu-se utilizar o formato **CSV,** por ser um formato simples e amplamente suportado, que é facilmente integrável em ferramentas de análise e folhas de cálculo. Para isso, também se optou por usar uma biblioteca (**OpenCSV**), para então simplificar a escrita de ficheiros deste tipo.

Testes Unitários

Para suportar testes unitários, foi integrada a biblioteca **JUnit 5**, permitindo validar componentes críticos através de asserções, contribuindo para maior fiabilidade e deteção precoce de regressões.

Interface e Visualização:

Para o desenvolvimento da interface gráfica do utilizador, escolhemos usar **JavaFX**, que permite maior personalização e flexibilidade na construção da GUI, suportando uma apresentação estruturada dos resultados.

Gestão de Dependências

Para o controlo e gestão de dependências, escolheu-se utilizar o **Maven,** que permite centralizar a configuração do projeto e automatizar a inclusão e atualização das bibliotecas no ficheiro *pom.xml*, simplificando a resolução de conflitos e reduzindo o esforço de manutenção.

Controlo de Versões:

Para realizar o controlo de versões e gestão colaborativa do código, decidiu-se utilizar a ferramenta **GitHub**, que permite gestão do histórico de alterações e suportar o trabalho em equipa, podendo assim ser controlado o ciclo de desenvolvimento da aplicação.

5.2. Modelação UML

A modelação do sistema recorreu à linguagem UML (*Unified Modeling Language*) para representar visualmente as funcionalidades, a estrutura estática e a arquitetura física da aplicação.

5.2.1. Diagrama de Casos de Usos

O **Diagrama de Casos de Uso**, representado na figura seguinte, ilustra as interações funcionais entre o utilizador e o sistema.

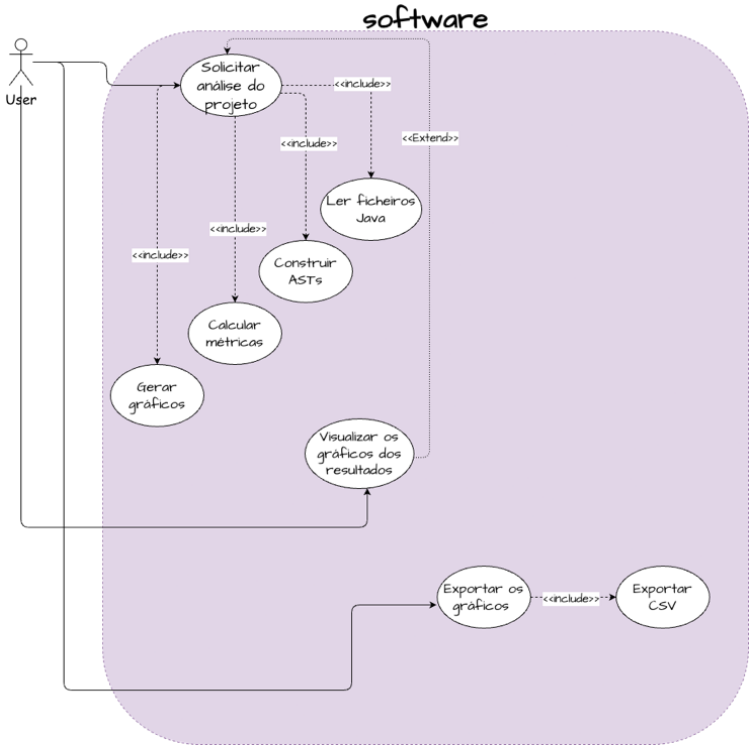


Figura 10. Diagrama de Casos de Uso

O ator principal é o **Utilizador** (*User*), que interage diretamente com o sistema (*software*) com o propósito de analisar um projeto Java e obter os resultados das métricas de qualidade.

A funcionalidade principal é "Solicitar análise do projeto", que engloba o processo de análise do código. Este processo engloba as etapas automáticas, representadas por relações de <<*include*>>, nomeadamente:

- **Ler ficheiros Java** – o sistema percorre o projeto e identifica os ficheiros *Java*;
- **Construir ASTs** – a partir dos ficheiros *Java*, o sistema cria a Árvore de Sintaxe Abstrata (AST) de cada ficheiro;
- **Calcular métricas** – com base nas ASTs, o sistema calcula as métricas de código;
- **Gerar Gráficos** – por fim, representa graficamente os resultados obtidos.

Uma vez concluído o processamento, o utilizador acede ao caso de uso "Visualizar os gráficos dos resultados". Esta etapa é fundamental para a interpretação dos dados, permitindo examinar detalhadamente as métricas calculadas para cada classe ou método.

Adicionalmente, o sistema oferece a funcionalidade de "Exportar os gráficos". Esta opção estende as capacidades da ferramenta, permitindo ao utilizador armazenar os dados analisados fora da aplicação através da geração de um ficheiro em formato CSV.

5.2.2. Diagrama de Classes

Este **Diagrama de Classes** representa a estrutura principal da ferramenta de Análise de Métricas de Código, evidenciando as relações entre as diferentes classes utilizadas nas diversas etapas do processo.

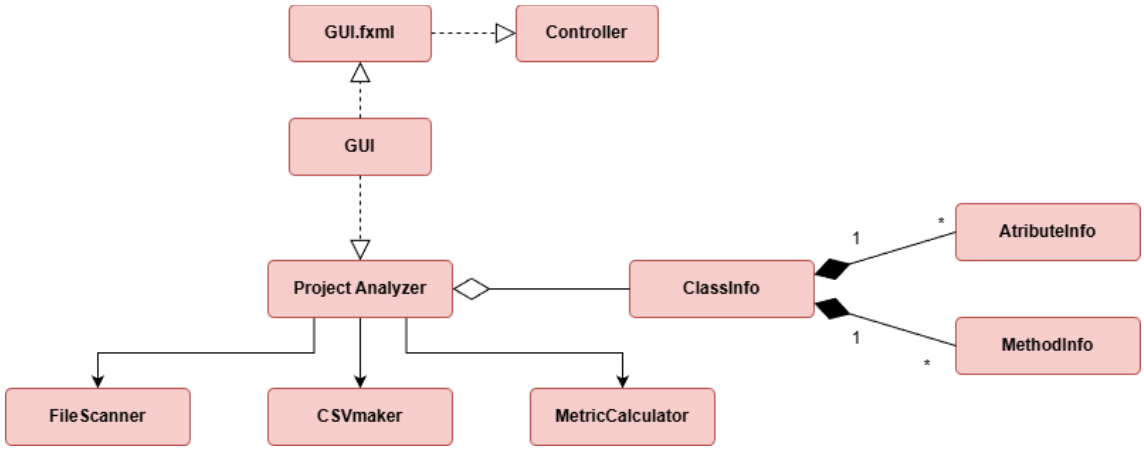


Figura 11. Diagrama de Classes Conceptual

A classe ***ProjectAnalyser*** é a classe central, ou seja, atua como uma "central de controlo" do sistema, coordenando o fluxo de dados entre os vários componentes:

1. **Camada de Interface** (GUI): Responsável pela interação com o utilizador. A classe GUI (cujo *layout* é definido via *FXML* e a lógica gerida pelo *Controller*) recolhe os *inputs* (como a seleção da diretoria) e invoca o *Project Analyzer* para iniciar o processamento, recebendo posteriormente os dados para atualizar a tabela e o gráfico.

2. **Camada de Serviços**: O *ProjectAnalyzer* delega tarefas específicas em classes especializadas:
    a. *FileScanner*: Responsável por analisar a pasta selecionada, identificar os ficheiros *.java* e construir as ASTs de cada um desses ficheiros;
    b. *MetricCalculator*: Responsável por calcular as métricas das diferentes classes e métodos através das ASTs criadas anteriormente;
    c. *CSVMaker*: Responsável por criar um ficheiro CSV com os resultados obtidos.
3. **Estrutura de Dados**: Para guardar as informações obtidas de cada classe, o sistema utiliza a classe *ClassInfo*, que agrega objetos do tipo:
    a. *MethodInfo*: Armazena informações sobre os métodos;
    b. *AttributeInfo*: Guarda os dados dos atributos de cada classe.

    Esta composição permite representar hierarquicamente uma classe Java e os seus respetivos métodos e atributos, facilitando a manipulação e consulta dos dados recolhidos.

5.2.3. Diagrama de implementação

O **Diagrama de Implementação** ilustra a organização física do sistema e as dependências necessárias para o seu funcionamento. A figura seguinte apresenta a estrutura da Ferramenta de Análise de Métricas.

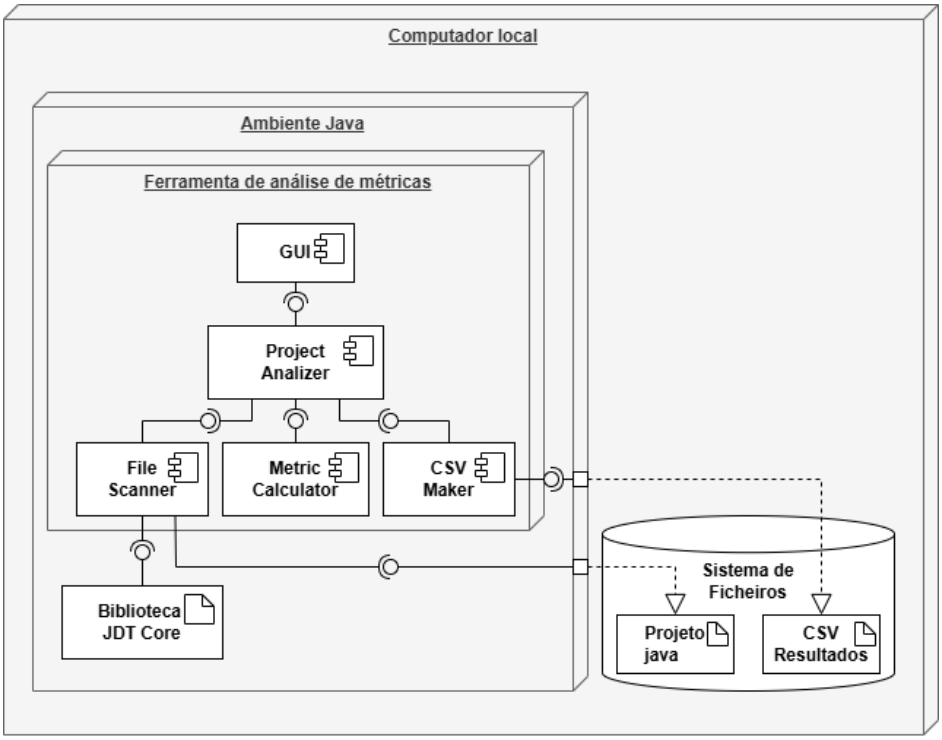


Figura 12. Diagrama de Implementação

A ferramenta é executada sobre o Ambiente Java (JRE). Esta camada isola a aplicação das especificidades do sistema operativo, garantindo o cumprimento do requisito de **Portabilidade** e permitindo que o software funcione em *Windows*, *macOS* ou *Linux* sem alterações.

Ao nível das interações com o exterior, destacam-se duas dependências fundamentais:

- **Biblioteca JDT Core:** Uma dependência essencial integrada no sistema para realizar a leitura e análise do código fonte.
- **Sistema de Ficheiros:** A interação direta com as pastas locais para ler o Projeto Java e gravar os resultados no ficheiro CSV.

Já internamente, o diagrama evidencia a organização lógica do sistema através das suas dependências hierárquicas: a interface gráfica (**GUI**) depende do controlador central (***Project Analyzer***), que por sua vez coordena os módulos de serviço (***File Scanner***, ***Metric Calculator*** e **CSVmaker**). Esta disposição reflete a separação de responsabilidades que fundamenta a arquitetura de software descrita na secção seguinte.

5.3. Arquitetura do Software

A seleção da arquitetura de software é um passo crucial para garantir a escalabilidade, a fácil manutenção e a clareza do sistema. Para o desenvolvimento da ferramenta de análise de métricas, optou-se por uma arquitetura que combina a separação de responsabilidades do padrão ***Model-View-Controller*** (MVC) com o isolamento lógico da **Arquitetura em Camadas** (*Layered Architecture*).

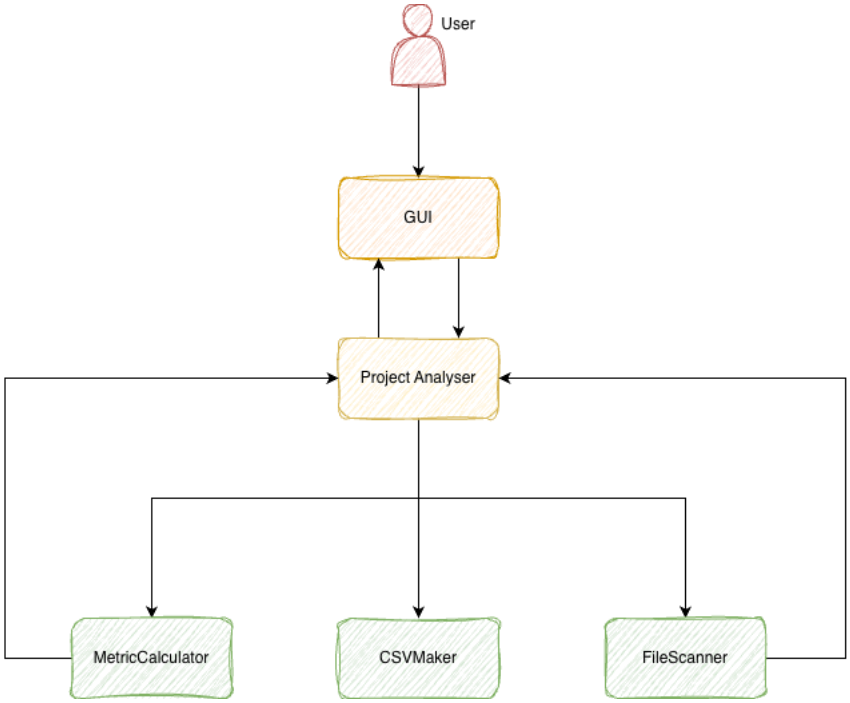


Figura 13. Esquema da arquitetura do software da ferramenta

Arquitetura em Camadas

A aplicação está estruturada em três camadas bem definidas, nas quais existe uma dependência estritamente unidirecional (as camadas superiores dependem das inferiores). Essas camadas são:

- **Interface (View/GUI):** Camada de topo, responsável pela apresentação de dados e interação do utilizador;
- **Controlo (Controller/ProjectAnalyser):** Camada intermédia, responsável por gerir o fluxo de trabalho e ligar a Interface aos Serviços;
- **Serviços (Model/Lógica de Negócio):** Camada inferior, onde residem as funcionalidades principais do sistema.

Esta separação clara de responsabilidades leva ao isolamento das camadas, o que faz com que alterações realizadas numa camada, não afetem a lógica das restantes. A separação permite também uma melhor organização do sistema, que simplifica a manutenção de funcionalidades específicas.

Padrão Model-View-Controller (MVC)

O padrão MVC é aplicado sobre a estrutura em camadas para alcançar a separação de preocupações:

- **View**: A *GUI* representa o que o utilizador vê e interage, sendo a única responsável por apresentar os resultados.
- **Model**: As classes *MetricCalculator*, *CSVmaker* e *FileScanner* (camada de Serviços) atuam como o Model, lidando com o processamento de dados, a lógica de cálculo e a exportação CSV.
- **Controller**: O *ProjectAnalyser* atua como a camada central, recebendo os pedidos da *View* e orquestrando as chamadas sequenciais aos *Serviços* para satisfazer esses pedidos.

Padrão Master-Slave (Coordenação)

O *ProjectAnalyser* assume o papel de componente **Mestre**, que inicia, coordena e consolida os resultados dos componentes **Escravos** (Serviços). Esta organização é particularmente adequada para o processo de análise, que é sequencial e de propósito único.

A escolha de uma **arquitetura monolítica** que adota padrões MVC e em Camadas é justificada pela natureza e escala do projeto. A combinação destes padrões permite que todos os componentes estejam interconectados e operem sobre o mesmo projeto, sendo compilados como uma única unidade executável, enquanto se beneficia da organização e da fácil manutenção consequentes da clara separação de responsabilidades.

6. Implementação

Neste capítulo vamos apresentar o desenvolvimento do projeto, dividindo entre:

1. **Ambiente de Desenvolvimento**, onde vamos falar sobre o IDE utilizado, linguagem e dependências;
2. **Integração da API JDT AST**, onde vamos descrever a API e como a utilizamos na implementação;
3. **Recolha de Métricas e Processamento**, onde vamos descrever como as métricas são recolhidas e a exportação das mesmas para um ficheiro CSV;
4. **Paralelismo entre a Implementação e a Arquitetura**, onde vamos descrever como implementamos a arquitetura descrita no ponto 5.3.

6.1. Ambiente de Desenvolvimento

Como foi referido anteriormente, o projeto foi desenvolvido no **IntelliJ** (IDE) utilizando a linguagem ***Java***.

Para o desenvolvimento do projeto e para a análise do código fonte, foram utilizadas as seguintes dependências:

- **org.junit.jupiter**: para permitir o desenvolvimento e execução de testes unitários com JUnit 5;
- **org.eclipse.jdt.core**: biblioteca principal, que permite criar, analisar e modificar ASTs;
- **org.openjfx**: utilizada para criar a GUI do trabalho com base em JavaFX;
- **com.opencsv**: utilizada para exportar os dados para ficheiros em formato CSV.

6.2. Integração da API JDT AST

A API JDT AST permite a análise do código fonte Java. O seu principal output é a **Árvore de Sintaxe Abstrata (AST)**, uma representação hierárquica e estruturada do código. Cada nó na árvore corresponde a uma construção sintática, como uma classe, um método, uma variável, ou uma instrução *if*.

O processo de gerar ASTs (*parsing*) é gerido pela classe *FileScanner* que utiliza o objeto *ASTParser* da API JDT:

1. São identificados todos os ficheiros .java do projeto a analisar;
2. Para cada ficheiro encontrado, é lido o conteúdo do mesmo (*parsing*), e criada uma *compilation unit* (AST);

Para a correta configuração do *ASTParser*, é necessário definir o ambiente de análise através do *classpath* e do *sourcepath*. O *sourcepath* indica ao *parser* a localização do código-fonte a analisar, enquanto o *classpath* estabelece o contexto das bibliotecas e dependências do projeto. Adicionalmente, é crucial habilitar a resolução de *bindings*, uma funcionalidade que permite ligar as referências de código aos seus tipos reais, sendo indispensável para o cálculo de métricas de acoplamento e herança, como o CBO e o DIT.

No contexto deste projeto, as *CompilationUnits* são utilizadas na classe *MetricCalculator*, que tem como objetivo percorrê-las para calcular as métricas de código e gerar os objetos *ClassInfo* a fim de armazenar os resultados obtidos.

6.3. Recolha de Métricas e Processamento

Nesta parte, vamos falar sobre o processo e recolha de métricas realizado pela aplicação:

Leitura dos ficheiros e criação de ASTs

O processo começa com a classe *FileScanner*, que é responsável por identificar os ficheiros Java e gerar as ASTs correspondentes.

Para percorrer a pasta do projeto e selecionar os ficheiros *.java*, é utilizada a **API Java NIO.2** (java.nio.file). Com esta API, é gerada uma ***Stream*** com os ficheiros, e percorremos a mesma para verificar quais destes são *.java*.

A seguir, para cada ficheiro identificado, é configurado um *ASTParser* e criada uma *CompilationUnit* (AST) com a **API JDT AST**.

Cálculo e armazenamento das Métricas

Após serem geradas as ASTs, o *MetricCalculator* recebe a lista de *CompilationUnits*. Esta classe vai percorrê-las utilizado o ASTVisitor e, em cada função de cálculo, à medida que visita nós relevantes, como métodos (*MethodDeclaration*) e atributos (*FieldDeclaration*), calcula a métrica correspondente (LOC, NOM, NOA, DIT, NOC, CBO, CC, NPRM ou Avg LOC/Método).

Para guardar os resultados a classe cria objetos *ClassInfo* (e, internamente, *MethodInfo* e *AtributeInfo*) para cada uma das ASTs.

A lista final de objetos *ClassInfo* é armazenada no *ProjectAnalyser* através da variável *classInfos*, ficando acessível para apresentação ou exportação.

Exportação para CSV

Caso o utilizador queira, os resultados podem ser exportados para um ficheiro CSV, função tratada pela classe *CSVmaker*.

Esta classe recebe a lista de *ClassInfo* e o caminho onde o utilizador quer o ficheiro. O ficheiro é construído no formato de uma tabela, onde, para automatizar a formatação em programas de folha de cálculo com configurações regionais europeias (como o Excel em Portugal), o *CSVWriter* da biblioteca **com.opencsv** foi configurado explicitamente para usar o ponto e vírgula (;) como separador de campos.

Assim, para cada um dos elementos da lista, a classe vai buscar os valores e insere no ficheiro.

Por fim, as 3 classes que realizam os pedidos (*FileScanner*, *MetricCalculator* e *CSVmaker*) são geridas pelo *ProjectAnalyser*, que comunica os resultados à GUI para o utilizador os poder visualizar.

6.4. Paralelismo entre a Implementação e a Arquitetura

A arquitetura da aplicação reflete o que foi descrito anteriormente na secção 5.3, tendo sido implementados os padrões de arquitetura ***Model-View-Controller*** **(MVC)** e **Arquitetura em Camadas.** Esta implementação garante uma separação clara de responsabilidades, facilidade de manutenção e escalabilidade.

Desta forma, as classes implementadas dividem-se em 3 camadas com responsabilidades distintas:

- **GUI**, que apresenta os dados ao utilizador e recolhe os pedidos do mesmo;
- **ProjectAnalyser**, que faz a gestão do fluxo de trabalho, ligando a GUI às classes que executam os pedidos do utilizador;
- **FileScanner, MetricCalculator e CSVmaker**, que executam a lógica implementada como a leitura dos ficheiros e a análise dos mesmos.

Com isto, podemos analisar o paralelismo entre a implementação e a arquitetura e justificar o seu uso:

Separação de Preocupações (MVC):

- A **GUI (View)** lida exclusivamente com a interação e visualização;
- As classes ***FileScanner***, ***MetricCalculator*** **e** ***CSVmaker***, que atuam como a camada ***Model*** (Serviços), lidam apenas com a manipulação dos dados, como calcular as métricas ou exportar o CSV, sem se preocupar com a visualização dos mesmos.
- O ***ProjectAnalyser*** **(*Controller*)** atua como a central de controlo da lógica da aplicação, recebendo pedidos da *View* e realizando uma sequência de chamadas às classes da camada de *Serviços*;

Arquitetura em Camadas (Isolamento de Componentes):

A arquitetura em camadas foi seguida, permitindo uma **dependência unidirecional** entre os componentes. Isto significa que cada camada conhece e depende apenas da camada que lhe está diretamente abaixo para executar os seus serviços, mas é completamente **independente** das camadas superiores.

Isto é demonstrado pela seguinte hierarquia de dependências:

- A ***GUI*** (Camada Superior) conhece apenas o ***ProjectAnalyser***, dependendo dele para poder apresentar os resultados dos pedidos do utilizador, sendo ela a responsável por ir buscar as conclusões da análise;
- O ***ProjectAnalyser*** (Camada Intermédia) conhece apenas as classes da ***camada de serviços***, dependendo das mesmas para guardar os resultados.
- Já na **camada *Model***, as suas classes não dependem de outras camadas para executar as suas tarefas.

Esta arquitetura tem como consequências:

- **Facilidade de atualização e manutenção:** Se for necessário, por exemplo, mudar a tecnologia da interface, a lógica de *MetricCalculator* permanece inalterada.
- **Facilidade de Teste:** O isolamento das classes de Serviços permite que o *FileScanner*, o *MetricCalculator* e o *CSVmaker* sejam submetidos a testes unitários de forma independente, sem a necessidade de simular a GUI ou o *controller*.

Padrão Master-Slave (ProjectAnalyser):

O *ProjectAnalyser* funciona como o componente **Mestre**, que inicia, coordena e junta os resultados dos componentes **Escravos** (Serviços).

Em resumo, a implementação em camadas e o uso do *ProjectAnalyser* como uma central de controlo garantem que cada componente se concentra na sua responsabilidade principal, resultando num **sistema fácil de atualizar, testar e realizar manutenção**.

7. Testes e Validação

A validação do sistema foi essencial para garantir que a ferramenta produz resultados corretos e consistentes e que os requisitos do projeto são efetivamente cumpridos. Num contexto de análise automática de código, pequenas falhas de *parsing*, contagem ou agregação podem comprometer a fiabilidade das métricas e, consequentemente, a utilidade da ferramenta.

Por esses motivos, foram adotadas duas abordagens complementares de teste e validação do sistema: a realização de testes unitários, para verificar de forma isolada os componentes críticos da aplicação, e a execução de cenários de qualidade alinhados com a *Utility Tree*, de modo a avaliar o cumprimento dos atributos de qualidade definidos para o sistema.

7.1. Testes unitários

Como referido acima, foram desenvolvidos testes unitários com o objetivo de validar, de forma isolada, os componentes mais críticos da aplicação, reduzindo o risco de regressões e aumentando a confiança na correção dos resultados produzidos. Em particular, os testes incidiram sobre classes ligadas ao fluxo principal do sistema: *FileScanner*, *MetricCalculator* e *CSVMaker*.

Os testes foram implementados com JUnit 5, recorrendo a diretórios temporários e a ficheiros Java de teste para simular diferentes cenários de utilização e garantir reprodutibilidade.

No final da execução, todos os testes unitários definidos foram executados com sucesso, não se registando falhas.

FileScannerTest

Os testes da classe *FileScanner* têm como objetivo validar e garantir que a classe é capaz de identificar corretamente os ficheiros *.java*, construir AST válidas e navegar nas mesmas.

Foram criados diretórios temporários com ficheiros artificiais e realizados os seguintes testes:

- **Filtragem de ficheiros Java (*testGetJavaFiles*):** garante que apenas ficheiros com extensão *.java* são considerados e que a contagem corresponde ao esperado no diretório de teste.
- **Diretório sem ficheiros Java (*testNoJavaFiles*):** valida que, quando não existem ficheiros *.java*, o resultado é uma lista vazia.
- **Geração de AST (*testParseFilesAST*):** confirma que é gerada uma *CompilationUnit* por ficheiro *.java* e que nenhuma unidade é nula.
- **Deteção de métodos na AST (*testMethodDetection*):** percorre cada *CompilationUnit* com *ASTVisitor* e verifica que os métodos são detetados conforme o conteúdo dos ficheiros de teste.
- **Resolução de bindings (*testBindings*):** valida que o *resolveBinding*() em *MethodDeclaration* devolve um *IMethodBinding* não nulo, indicando que a AST suporta resolução de ligações necessária para métricas mais avançadas.

MetricCalculatorTest

Os testes da classe *MetricCalculator* têm como objetivo validar que a classe calcula corretamente as métricas estipuladas (LOC, NOM, NOA, DIT, NOC, CBO, MCC/CC, NPRM e AvgLOCMethod), a partir das ASTs geradas com base em ficheiros Java de teste controlados.

Foram criados ficheiros Java temporários (A, B, C e D) com estruturas específicas, permitindo comparar os resultados obtidos com valores esperados. Foram realizados os seguintes testes:

- **Criação do *calculator* e deteção de classes (*testMetricCalculatorExists*):** confirma que o *MetricCalculator* é instanciado, que a lista de classes não é nula, que são detetadas 4 classes (A, B, C, D) e que cada uma existe no conjunto produzido.
- **Contagem de métodos e atributos (*testClassInfo*):** valida NOM e NOA para cada classe.
- **Complexidade ciclomática (*testCalculateMCC*):** verifica o cálculo da complexidade ciclomática com estruturas de controlo conhecidas.
- **Linhas de código (*testCalculateLOC*):** valida a métrica LOC com base nas linhas inicial/final de cada classe.
- **Profundidade de herança (*testCalculateDIT*):** valida DIT para classes com cadeia de herança.
- **Número de filhos (*testCalculateNOC*):** valida NOC.
- **Média de LOC por método (*testAvgLocMethod*):** valida AvgLOCMethod, incluindo o caso de classes sem métodos.
- **Acoplamento entre objetos (*testCalculateCBO*):** valida CBO principalmente via herança.
- **Média de parâmetros por método (*testCalculateNPRM*):** valida NPRM com métodos de assinatura conhecida.

CSVMakerTest

Os testes da classe *CSVMaker* têm como objetivo validar que a classe cria corretamente o ficheiro CSV (*resultado_da_analise.csv*), garantindo a presença do cabeçalho e a escrita consistente dos dados das classes e respetivas métricas.

Mais uma vez, os testes recorrem a uma pasta temporária para assegurar isolamento e evitar efeitos colaterais e foram preparadas instâncias de *ClassInfo* com configurações distintas (classes com e sem métodos/atributos) para simular diferentes resultados de análise. Foram realizados os seguintes testes:

- **CSV vazio (*testEmptyCSV*):** gera um CSV a partir de uma lista vazia de classes e verifica que: o ficheiro é criado, que existe apenas uma linha no CSV (o cabeçalho) e que o mesmo corresponde exatamente ao formato esperado.
- **CSV com uma classe (*testCSVComUmaClasse*):** gera o CSV com uma única classe e valida que: existem duas linhas (cabeçalho e dados da classe), que a linha de dados tem 10 colunas e que o nome da classe na primeira coluna é o esperado.
- **CSV com várias classes (*testCSVComVariasClasses*):** gera o CSV com três classes e valida que: existem quatro linhas (cabeçalho e 3 linhas de dados) e que os nomes das classes surgem no ficheiro e na ordem prevista.

7.2. Testes dos atributos de qualidade

Como referido anteriormente, para além dos testes unitários, foram realizados testes orientados aos atributos de qualidade definidos na *Utility Tree,* de forma a verificar, através

de cenários concretos, se o sistema cumpre as características consideradas críticas para o projeto.

Estes testes foram conduzidos como validações do comportamento do sistema em utilização, tendo como base os cenários priorizados e os respetivos critérios de aceitação. No final da execução, todos os cenários foram aprovados, não se registando incumprimentos dos atributos de qualidade definidos, o que confirma o alinhamento da solução com as características consideradas críticas para o projeto.

Testes de usabilidade

A usabilidade foi avaliada por ser crítica para garantir que a ferramenta pode ser utilizada de forma eficaz por novos utilizadores, e com resultados facilmente interpretáveis.

Facilidade de Aprendizagem (H, L) - Como foi testado e Resultados:
Foi pedido a um utilizador (com conhecimentos de engenharia de *software*) sem contacto prévio com a ferramenta que executasse o fluxo principal (selecionar projeto, analisar, visualizar resultados, exportar CSV) sem consulta de documentação.

Verificou-se que o utilizador conseguiu completar todo o fluxo sem assistência e identificou todos os controlos principais da aplicação compreendendo todas as mensagens apresentadas pelo sistema, pelo que se pôde concluir que a interação foi intuitiva e fácil de compreender.

Clareza da Visualização (H, M) - Como foi testado e Resultados:
No mesmo teste com o utilizador, foi avaliada a forma como os resultados são apresentados após a análise. Confirmou-se que as métricas são exibidas de forma organizada e legível, com correspondência clara entre cada classe analisada e os respetivos valores.

Conclui-se, portanto, que a visualização suporta uma interpretação rápida e sem ambiguidade dos resultados.

Testes de Modificabilidade

A modificabilidade foi testada para confirmar que a arquitetura permite evoluir o sistema com esforço controlado, nomeadamente na introdução de novas métricas e na alteração da forma de visualização, sem impacto indevido em componentes não relacionados.

Ao invés de se alterar efetivamente o código para adicionar novas métricas e alterar a visualização, foi simulado o percurso que se teria de fazer para fazer essas alterações, identificando os pontos de extensão previstos e verificando se a alteração poderia ser executada de forma localizada e coerente com a separação de responsabilidades definida.

Extensibilidade (M, H) - Como foi testado e Resultados:
Para validar a extensibilidade, simulou-se a adição de uma nova métrica seguindo o percurso esperado de implementação. Concluiu-se que para adicionar uma nova métrica é necessário:

- Definir a métrica e a sua fórmula, identificando a informação necessária a partir da AST.

- Identificar o ponto de integração no *MetricCalculator*, criando uma função para o cálculo da métrica e garantir a sua invocação no fluxo de cálculo.
- Adicionar a métrica ao modelo de dados (*ClassInfo*, criando o atributo correspondente e respetivos *getters*/*setters*), mantendo o impacto controlado e explícito.
- Propagar a métrica para a apresentação/exportação, adicionando uma nova coluna na tabela da GUI e gráfico e uma nova coluna no CSV.
- Por fim de forma a garantir a correção da solução, devem-se atualizar os testes unitários do *MetricCalculator* e do *CSVmaker*, garantindo validação automática do novo valor.

Concluiu-se que a arquitetura permite introduzir uma nova métrica de forma localizada (principalmente no *MetricCalculator* e no modelo *ClassInfo*), exigindo apenas ajustes pontuais na camada de apresentação/exportação para exposição do valor, confirmando um bom nível de extensibilidade.

Flexibilidade da UI (L, L)- Como foi testado e Resultados:
Para validar a flexibilidade da interface, simulou-se a alteração da forma de visualização dos resultados. Verificou-se que estas alterações podem ser realizadas atuando apenas sobre a camada de apresentação, nomeadamente o ficheiro *GUI.fxml* (componentes *JavaFX* responsáveis pela tabela/visualização) e o respetivo *Controller*, onde se adiciona ou ajusta a lógica de interação necessária.

Conclui-se que a alteração da visualização não afeta a lógica de cálculo, confirmando a separação entre a camada de apresentação e a camada de processamento. A modificação restringe-se apenas à GUI, o que evidencia flexibilidade na evolução da interface.

Testes de Fiabilidade

A fiabilidade foi avaliada para garantir que a ferramenta se mantém estável durante análises prolongadas ou em projetos com maior volume de ficheiros, evitando encerramentos inesperados e assegurando tratamento adequado de erros.

Disponibilidade (H, M) - Como foi testado e Resultados:
A aplicação foi executada em projetos com alguma dimensão e repetiu-se a análise em múltiplas execuções, observando-se estabilidade e comportamento perante situações anómalas (por exemplo ficheiros que não eram do tipo correto).

Verificou-se que a ferramenta não terminou inesperadamente, mesmo quando existem problemas nos ficheiros, apresentando mensagens claras e mantendo-se operativo.

Testes de Portabilidade

A portabilidade foi verificada para assegurar que a ferramenta pode ser executada em diferentes sistemas operativos sem alterações ao código, tirando partido da JVM e garantindo comportamento equivalente entre ambientes.

Adaptabilidade de SO (H, L) - Como foi testado e Resultados:
A validação deste atributo foi facilitada pelo facto de o grupo integrar elementos com *Windows* e *macOS*.

Assim, para testar a adaptabilidade, a ferramenta foi executada em ambos os sistemas, repetindo-se o fluxo principal. Verificou-se que o comportamento foi equivalente nos dois ambientes e que todas as funcionalidades operaram corretamente sem necessidade de qualquer alteração ao código-fonte. Concluindo-se, portanto, que a aplicação assegura a adaptabilidade entre sistemas operativos, conforme esperado.

8. Resultados e Avaliação

Após a validação do sistema através dos testes descritos na secção anterior, apresentam-se agora os resultados obtidos e a respetiva avaliação. Começa-se por apresentar o BPMN do fluxo principal, seguido de evidências da execução da ferramenta. Por fim, comparam-se as métricas obtidas com as de outras ferramentas, de modo a avaliar a consistência dos resultados.

8.1. Enquadramento do Processo

O diagrama BPMN é apresentado para sintetizar, de forma visual, o fluxo principal da ferramenta, servindo como referência para interpretar as evidências de execução (apresentadas de seguida) e confirmar que o comportamento observado segue o processo definido.

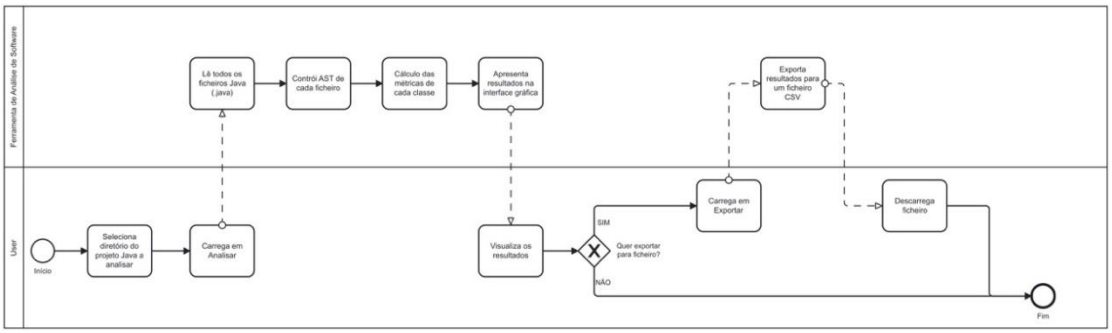


Figura 14. Diagrama BPMN

Neste diagrama está então representado o fluxo do processo de análise de código, desde o momento em que o utilizador seleciona o projeto a analisar até à possível descarga do ficheiro com os resultados.

O processo está dividido em duas "*swimlanes*", o **utilizador** (*User*), que interage com a interface gráfica do sistema e executa ações, e o **sistema** (*Ferramenta de Análise do Software*), que realiza as ações pedidas pelo utilizador e gera os resultados.

Este fluxo inicia-se quando o utilizador seleciona o diretório do projeto Java a analisar e clica no botão "Analisar", para então ser iniciada a análise. A partir desse ponto, o sistema assume o controlo do processo e executa, de forma sequencial, as etapas nucleares da análise: leitura dos ficheiros *.java*, construção da AST para cada ficheiro e cálculo das métricas por classe. Concluída a análise, a ferramenta apresenta os resultados na interface gráfica, permitindo ao utilizador consultar a informação calculada.

Por fim, o utilizador pode escolher exportar ou não os dados para um ficheiro CSV. Caso não queira, o processo termina após a visualização dos resultados. Caso contrário, o

utilizador aciona a opção de exportação; o sistema prepara os dados e gera o ficheiro CSV, concluindo o fluxo com a descarga do ficheiro com os resultados.

8.2. Execução da ferramenta

De seguida, apresentam-se evidências de execução da ferramenta, correspondentes às principais etapas do processo de análise e exportação.

Quando se corre a aplicação surge o ecrã inicial da mesma, onde o utilizador dispõe das ações principais: "*Escolha o Projeto Java*" e "*Analisar o Codigo*", dando início ao fluxo representado no diagrama anterior.

Ao clicar no botão de escolha, o utilizador é direcionado para o seu sistema de ficheiros onde pode escolher o projeto que quer analisar. Ao clicar no botão de análise, é iniciada a análise do projeto escolhido.

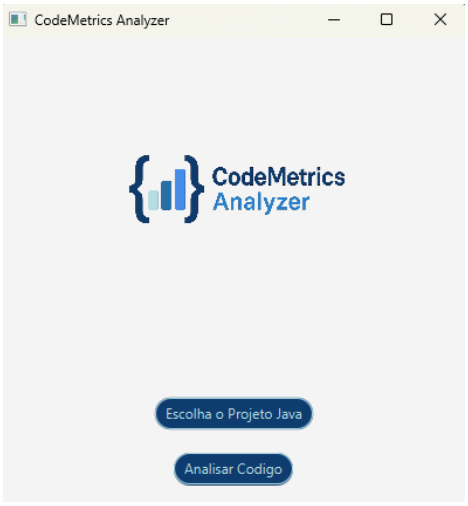


Figura 15. Página inicial da ferramenta

Ao tentar executar a análise sem selecionar previamente um projeto, o sistema interrompe o fluxo e apresenta uma mensagem de erro.

A mesma situação acontece após a seleção de um diretório que não contém código Java analisável, o sistema identifica a condição e notifica o utilizador.

Estas validações garantem que o processo só avança quando existe um input válido, prevenindo falhas e melhorando a experiência de utilização.

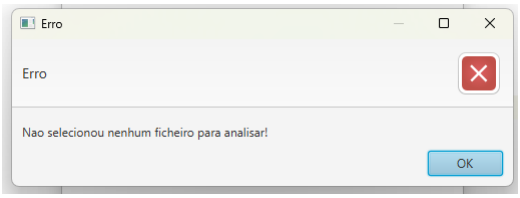


Figura 16. Erro apresentado ao tentar analisar e nenhum ficheiro foi selecionado

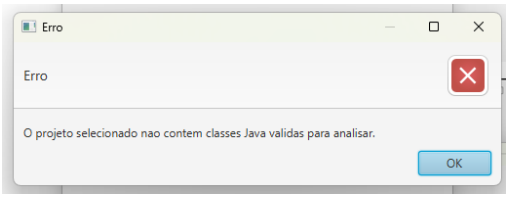
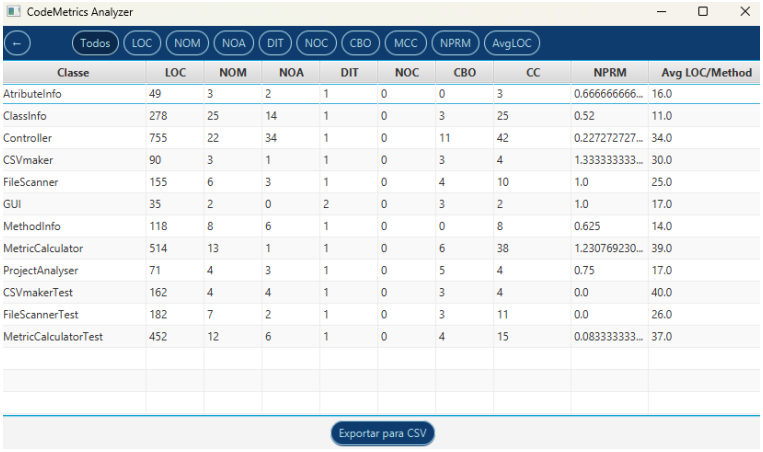


Figura 17. Erro apresentado ao tentar analisar um projeto/pasta do tipo incorreto

Quando é selecionado um ficheiro válido, o sistema calcula as métricas para o mesmo, disponibilizando os resultados numa tabela.



Classe	LOC	NOM	NOA	DIT	NOC	CBO	CC	NPRM	Avg LOC/Method
AtributeInfo	49	3	2	1	0	0	3	0.666666666...	16.0
ClassInfo	278	25	14	1	0	3	25	0.52	11.0
Controller	755	22	34	1	0	11	42	0.227272727...	34.0
CSVmaker	90	3	1	1	0	3	4	1.333333333...	30.0
FileScanner	155	6	3	1	0	4	10	1.0	25.0
GUI	35	2	0	2	0	3	2	1.0	17.0
MethodInfo	118	8	6	1	0	0	8	0.625	14.0
MetricCalculator	514	13	1	1	0	6	38	1.230769230...	39.0
ProjectAnalyser	71	4	3	1	0	5	4	0.75	17.0
CSVmakerTest	162	4	4	1	0	3	4	0.0	40.0
FileScannerTest	182	7	2	1	0	3	11	0.0	26.0
MetricCalculatorTest	452	12	6	1	0	4	15	0.083333333...	37.0

Figura 18. Resultados para a análise

Caso o utilizador o pretenda, pode também para cada métrica, visualizar graficamente os resultados, selecionando os botões no topo do ecrã que o indicam.

Neste caso, na imagem abaixo, foi selecionada a métrica NOM.

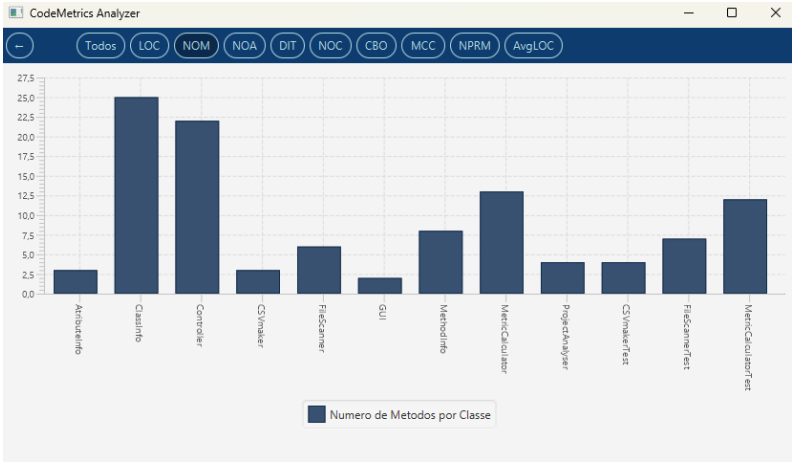


Figura 19. Visualização gráfica da métrica NOM por classe

Quando o utilizador seleciona o botão “Exportar para CSV” no ecrã de resultados, a aplicação apresenta uma janela para escolha do diretório de destino. Após a confirmação, o sistema gera o ficheiro CSV na pasta selecionada e apresenta uma mensagem de sucesso ou, em caso de erro, um aviso de falha, garantindo feedback imediato sobre o resultado da operação.

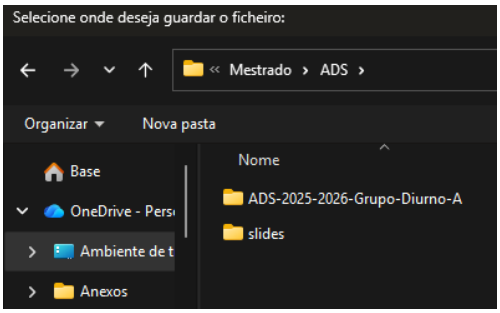


Figura 20. Pasta selecionada para a criação do CSV

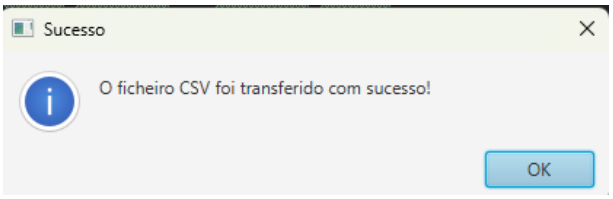


Figura 21. Alerta de sucesso na criação do ficheiro

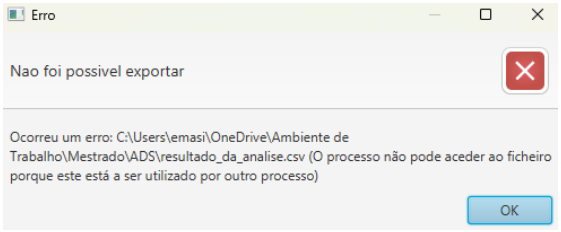


Figura 22. Alerta de erro

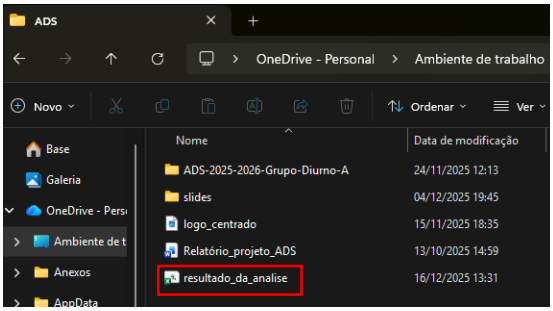
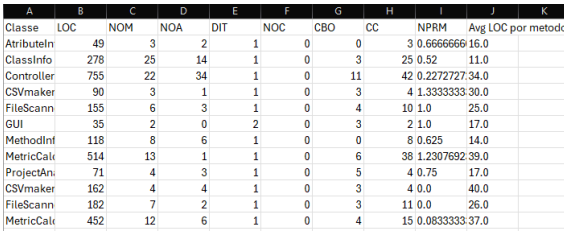


Figura 23. Pasta com o ficheiro criado



Classe	LOC	NOM	NOA	DIT	NOC	CBO	CC	NPRM	Avg LOC por metodo
AtributeIn	49	3	2	1	0	0	3	0.6666666	16.0
ClassInfo	278	25	14	1	0	3	25	0.52	11.0
Controller	755	22	34	1	0	11	42	0.2272727	34.0
CSVmaker	90	3	1	1	0	3	4	1.3333333	30.0
FileScann	155	6	3	1	0	4	10	1.0	25.0
GUI	35	2	0	2	0	3	2	1.0	17.0
MethodInf	118	8	6	1	0	0	8	0.625	14.0
MetricCal	514	13	1	1	0	6	38	1.2307692	39.0
ProjectAn	71	4	3	1	0	5	4	0.75	17.0
CSVmaker	162	4	4	1	0	3	4	0.0	40.0
FileScann	182	7	2	1	0	3	11	0.0	26.0
MetricCal	452	12	6	1	0	4	15	0.0833333	37.0

Figura 24. Conteúdo do ficheiro criado

Caso o utilizador pretenda analisar outro ficheiro, ao clicar no ícone da seta retorna para o ecrã inicial, onde pode então voltar a executar o processo.

8.3. Avaliação da Ferramenta

Após a demonstração do fluxo de execução, esta secção apresenta a avaliação da ferramenta com base nos resultados produzidos e na sua consistência.

Começa-se por sintetizar os outputs gerados numa execução real e, de seguida, procede-se a uma comparação com ferramentas externas de referência, resultando numa apreciação global do desempenho e adequação da solução face aos objetivos do projeto.

Resultados obtidos e qualidade do Código

Optou-se por testar a ferramenta utilizando o código do próprio projeto, permitindo não só validar o seu funcionamento num caso real, como também obter uma caracterização das métricas e, consequentemente, uma leitura inicial sobre a qualidade do código desenvolvido. Nesta subsecção apresentam-se, assim, os resultados obtidos nessa execução.

Tabela 3. Resultados das métricas para o Projeto

Classe	LOC	NOM	NOA	DIT	NOC	CBO	CC	NPRM	Avg LOC por metodo
AtributeInfo	49	3	2	1	0	0	3	0.6(6)	16.0
ClassInfo	278	25	14	1	0	3	25	0.52	11.0
Controller	755	22	34	1	0	11	42	~0.23	34.0
CSVmaker	90	3	1	1	0	3	4	1.3(3)	30.0
FileScanner	153	6	3	1	0	4	10	1.0	25.0
GUI	35	2	0	2	0	3	2	1.0	17.0
MethodInfo	118	8	6	1	0	0	8	~0.63	14.0
MetricCalculator	514	13	1	1	0	6	38	~1.23	39.0
ProjectAnalyser	71	4	3	1	0	5	4	0.75	17.0
CSVmakerTest	162	4	4	1	0	3	4	0.0	40.0
FileScannerTest	182	7	2	1	0	3	11	0.0	26.0
MetricCalculatorTest	452	12	6	1	0	4	15	~0.08	37.0

A partir dos resultados obtidos, é possível retirar algumas conclusões relevantes sobre complexidade, qualidade do nosso código e, indiretamente, sobre risco de manutenção. Observa-se uma distinção clara entre classes de lógica e classes de dados.

Complexidade

- *MetricCalculator* apresenta CC=38 para 13 métodos, sugerindo métodos com várias estruturas de controlo, o que é coerente com o cálculo das métricas.
- *Controller* (classe que trata das funções da GUI) tem CC=42 distribuída por 22 métodos, o que indica maior complexidade total, mas com lógica mais distribuída, típica de uma classe de orquestração.
- Classes de dados (*ClassInfo*, *MethodInfo*, *AtributeInfo*) mantêm complexidade baixa por método, indicando métodos simples (predominantemente getters/setters), o que favorece manutenção.

Acoplamento

- Em termos de acoplamento, a *Controller* apresenta o maior valor de CBO, o que é esperado dado que agrega componentes da interface (*JavaFX*).
- As restantes classes apresentam CBO baixo (valores reduzidos), sugerindo boa separação de responsabilidades.

Herança

- Em geral, DIT = 1 e NOC = 0, o que indica uma estrutura pouco dependente de herança e mais "plana". Isto tende a reduzir complexidade estrutural e facilitar a compreensão do código.
- Apenas a Classe da GUI tem DIT=2, já que estende *Application* (e*xtends Application*).

Classes de Teste

- As classes de teste apresentam frequentemente LOC elevado, mas com complexidade relativamente baixa, o que é normal: há mais linhas por repetição de cenários e *asserts*, mantendo-se a lógica simples e linear.

No conjunto, as métricas indicam uma estrutura saudável: a complexidade está concentrada onde é expectável (cálculo e orquestração), as classes de dados são simples e a arquitetura evita heranças desnecessárias.

Comparação da ferramenta - Avaliação

Para avaliar a consistência dos valores calculados pela ferramenta desenvolvida, procedeu-se à comparação com ferramentas externas de referência para métricas de código em projetos Java. A comparação foi efetuada sobre o mesmo projeto e versão (*Ferramenta de Análise de Código*), analisando as métricas ao nível da classe e discutindo eventuais discrepâncias resultantes de diferenças de definição ou de estratégia de cálculo adotadas por cada ferramenta.

MetricsTree

A primeira comparação foi realizada usando a ferramenta ***MetricsTree*** do *IntelliJ*, que permite analisar um projeto e obter, de forma hierárquica, um conjunto de métricas relevantes, quer para classes como para métodos. A figura seguinte apresenta um exemplo do resultado produzido para uma das classes analisadas.


Class: AtributeInfo

Me...	Metrics...	Description	Value	Regular...
CLOC	Statistics	Lines of Code	23	
CCC	G. Ann C...	Cognitive Complexity	0	[0..32)
CHVL	Halstead...	Halstead Volume	53,8887	
CHD	Halstead...	Halstead Difficulty	3,75	
CHL	Halstead...	Halstead Length	17	
CHEF	Halstead...	Halstead Effort	202,0827	
CHVC	Halstead...	Halstead Vocabulary	9	
CHER	Halstead...	Halstead Errors	0,0115	
WMC	Chidamb...	Weighted Methods Per Class	3	[0..12)
DIT	Chidamb...	Depth Of Inheritance Tree	1	[0..3)
CBO	Chidamb...	Coupling Between Objects	2	[0..14)
RFC	Chidamb...	Response For A Class	3	[0..45)
LCOM	Chidamb...	Lack Of Cohesion Of Methods	2	
NOC	Chidamb...	Number Of Children	0	[0..2)
NOA	Lorenz-K...	Number Of Attributes	2	[0..4)
NOO	Lorenz-K...	Number Of Operations	16	
NOOM	Lorenz-K...	Number Of Overridden Methods	0	[0..3)
NOAM	Lorenz-K...	Number Of Added Methods	2	
SIZE2	Li-Henry...	Number Of Attributes And Methods	18	

Figura 25. Excerto dos resultados para as métricas da Classe AtributeInfo

Como o *output* da ferramenta para cada classe contém bastantes métricas irrelevantes para a nossa comparação, com base nos outputs de cada classe foi construída a seguinte tabela:

Tabela 4. Resultados das métricas para o Projeto com MetricsTree

Classe	LOC	NOM	NOA	DIT	NOC	CBO	CC	NPRM	Avg LOC por metodo
AtributeInfo	23	3	2	1	0	2	NA	NA	NA
ClassInfo	166	25	14	1	0	6	NA	NA	NA
Controller	442	22	34	1	0	38	NA	NA	NA
CSVmaker	65	3	1	1	0	14	NA	NA	NA
FileScanner	103	6	3	1	0	15	NA	NA	NA
GUI	23	2	4	2	0	12	NA	NA	NA
MethodInfo	64	8	6	1	0	2	NA	NA	NA

MetricCalculator	408	13	1	1	0	38	NA	NA	NA
ProjectAnalyser	45	4	3	1	0	9	NA	NA	NA
CSVmakerTest	107	4	4	1	0	15	NA	NA	NA
FileScannerTest	134	7	2	1	0	26	NA	NA	NA
MetricCalculatorTest	334	12	6	1	0	20	NA	NA	NA

Comparando os resultados desta ferramenta e da nossa verificámos algumas métricas com resultados iguais e outras com algumas discrepâncias.

Observa-se uma **correspondência exata** nas métricas estruturais e de contagem de entidades: NOM (*Number of Methods*), NOA (*Attributes*), DIT (*Inheritance*) e NOC (Children), apresentam o mesmo resultado para estas métricas em ambas as ferramentas.

Isto valida a precisão do motor de *parsing* e da geração da AST da nossa aplicação. O sistema identifica corretamente todas as assinaturas de métodos, campos de classe e hierarquias de herança sem falhas.

Existem **diferenças notáveis** nas métricas de LOC e CBO. Estas discrepâncias não indicam erros, mas sim diferentes abordagens na definição das métricas:

- A nossa ferramenta apresenta consistentemente valores de LOC superiores aos da ferramenta de referência.
  A nossa ferramenta conta todas as linhas do ficheiro, incluindo espaços em branco e comentários. A ferramenta *MetricsTree* não conta com tudo isso, aparentando contar apenas com as linhas com conteúdo.

- Existe também uma variação significativa nos valores de acoplamento, sendo os calculados pela nossa ferramenta muito menores, isto porque o cálculo do CBO é sensível ao algoritmo utilizado.
  Na nossa ferramenta, métrica CBO (*Coupling Between Objects*) foi implementada contando o número de tipos distintos não primitivos aos quais cada classe está acoplada, com base em relações explícitas observáveis na AST.
  Em particular, o cálculo considera como fontes de acoplamento a Herança (através do *superclass type*, quando existe), atributos (contabilizando o tipo declarado de cada campo) e assinaturas de métodos, contabilizando o tipo de retorno, os tipos dos parâmetros e os tipos de exceções lançadas (*throws*).
  Para evitar inflacionar o resultado com elementos não relevantes do ponto de vista de acoplamento entre objetos, foram aplicadas regras de normalização e filtragem:
  - Tipos primitivos, *void* e *String*;
  - Tipos são contados apenas uma vez (tipos distintos), evitando duplicação;
  - Em casos de *arrays* e tipos parametrizados (como *List<A>*), o algoritmo extrai o tipo base, de forma a contabilizar o acoplamento ao tipo efetivo e manter consistência;
  - A própria classe não é contabilizada, removendo o seu identificador do conjunto final.

  Assim, é expectável que ferramentas como o *MetricsTree* apresentem valores superiores, caso incluam outras fontes de acoplamento (por exemplo, tipos usados no corpo dos métodos, chamadas a métodos de outras classes, importações, instâncias criadas localmente, anotações ou dependências indiretas). Deste

modo, as discrepâncias observadas refletem sobretudo diferenças de definição e escopo da métrica, e não necessariamente erros de cálculo.

Por fim notou-se também uma diferença no caso da classe GUI: a métrica NOA deu 4. A nossa implementação contabiliza apenas atributos de classe declarados diretamente no corpo da classe, identificados na AST através de *FieldDeclaration*. Como a classe GUI não declara quaisquer *fields* (contém apenas variáveis locais no método *start*() e parâmetros), o valor obtido é NOA = 0. O *MetricsTree* parece incluir as variáveis locais do método *start*(), resultando em NOA=4.

CodeMR

Para reforçar a avaliação da ferramenta e complementar análise anterior, foi realizada uma comparação adicional com o *CodeMR*, uma ferramenta externa de análise estática para projetos Java que permite obter métricas e visualizações a diferentes níveis de granularidade.

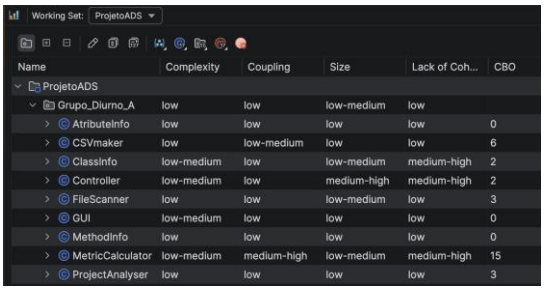


Figura 26. Parte dos resultados da ferramenta CodeMR

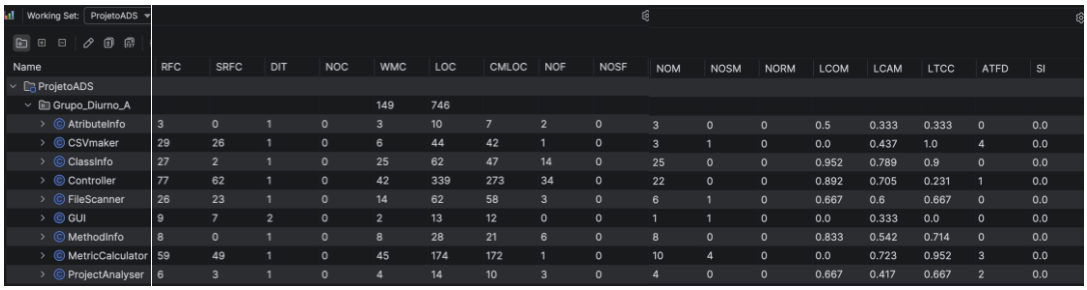


Figura 27. Resto dos resultados da ferramenta CodeMR

Como, mais uma vez, o output da ferramenta para cada classe contém bastantes métricas irrelevantes para a nossa comparação, com base nos outputs foi construída a seguinte tabela:

Tabela 5. Resultados das métricas para o Projeto com CodeMR

Classe	LOC	NOM	NOA	DIT	NOC	CBO	CC	NPRM	Avg LOC por metodo
AtributeInfo	10	3	NA	1	0	0	NA	NA	NA
ClassInfo	62	25	NA	1	0	2	NA	NA	NA
Controller	339	22	NA	1	0	2	NA	NA	NA
CSVmaker	44	3	NA	1	0	6	NA	NA	NA
FileScanner	62	6	NA	1	0	3	NA	NA	NA

GUI	13	1	NA	2	0	0	NA	NA	NA
MethodInfo	28	8	NA	1	0	0	NA	NA	NA
MetricCalculator	174	10	NA	1	0	15	NA	NA	NA
ProjectAnalyser	14	4	NA	1	0	3	NA	NA	NA

É de notar que esta ferramenta não calculou as métricas para as classes de testes, ao contrário das anteriores.

Comparando os resultados desta ferramenta e da nossa verificámos algumas métricas com resultados iguais e outras com algumas discrepâncias.

Observa-se uma consistência nas métricas estruturais e de contagem de entidades: NOM (*Number of Methods*), DIT (*Inheritance*) e NOC (*Children*), apresentam o mesmo resultado para estas métricas em ambas as ferramentas. A métrica NOM (número de métodos) coincide também na generalidade das classes, com exceção da classe GUI, onde a nossa ferramenta reporta 2 e o CodeMR reporta 1, por não considerar o método *main* na contagem.

Mais uma vez, as discrepâncias nas métricas LOC e CBO têm a ver com as diferentes definições operacionais. Neste caso a diferença no LOC é especialmente marcada, já que *CodeMR* aparenta contar apenas as linhas lógicas, ou seja, ignorando comentários e linhas em branco. E embora os números absolutos sejam distintos, a proporção mantém-se, as classes maiores numa ferramenta são também as maiores na outra.

Apesar de neste caso a diferença do CBO ser menos brusca, os valores distintos voltam a indicar que os critérios para o seu cálculo foram diferentes.

Conclusões

A comparação com o *Metrics Tree* e o *CodeMR* reforçou a fiabilidade da solução desenvolvida, evidenciando elevada concordância nas métricas estruturais (nomeadamente NOM, DIT, NOC e em geral NOA), o que valida o processo de *parsing* e exploração da AST.

As discrepâncias observadas principalmente no LOC e CBO são explicáveis por diferenças de definição e cálculo entre ferramentas, sem comprometer a interpretação relativa dos resultados, ou seja, as classes com valores elevados na nossa ferramenta surgem também destacadas nas ferramentas externas, confirmando a sua utilidade na deteção de pontos críticos.

Adicionalmente, a nossa ferramenta, tal como a *MetricsTree* mostrou boa cobertura ao analisar igualmente classes de teste, contribuindo para uma visão mais completa do projeto.

9. Conclusões e Trabalho Futuro

O desenvolvimento da **Ferramenta De Análise de Métricas de Código** permitiu alcançar com sucesso o objetivo principal do projeto: criar uma aplicação robusta capaz de analisar

automaticamente projetos Java e aferir a sua qualidade através de indicadores quantitativos.

A utilização da biblioteca **Eclipse JDT** revelou-se fundamental para a construção de Árvores de Sintaxe Abstrata (AST), permitindo uma extração precisa de informações estruturais do código. Do ponto de vista arquitetural, a adoção de uma estutura modular, com separação clara entre a interface (***GUI***), o controlo (***ProjectAnalyzer***) e os serviços (***MetricCalculator***, ***CSVmaker*** e ***FileScanner***), garantiu o cumprimento dos atributos de qualidade definidos, nomeadamente a **Modificabilidade** e **Portabilidade**.

A metodologia ***Scrum*** facilitou a gestão de tarefas e o acompanhamento do progresso, permitindo à equipa contornar dificuldades técnicas, especialmente na compreensão inicial da API JDT, e entregar um produto final dentro do prazo estipulado.

Concluiu-se, assim, que a ferramenta desenvolvida cumpre integralmente os requisitos funcionais propostos. A fiabilidade dos resultados foi corroborada tanto pela execução de testes unitários como pela comparação direta com ferramentas de referência (*MetricsTree* e *CodeMR*), validando a precisão do motor de análise. Desta forma, o sistema entregue não só responde ao problema inicial, como estabelece uma base arquitetural sólida e extensível para futuras evoluções.

Trabalhos Futuros

Apesar de a ferramenta cumprir os objetivos iniciais, identificaram-se várias oportunidades de melhoria e expansão que poderiam valorizar o produto numa próxima iteração:

- **Persistência de Dados e Histórico**: Atualmente, a ferramenta analisa o estado atual do projeto. Seria vantajoso integrar uma base de dados (como *SQLite*) para guardar os resultados de cada análise. Isto permitiria comparar a evolução do código ao longo do tempo, ajudando a, por exemplo, monitorizar o aumento da sua complexidade.
- **Expansão do Catálogo de Métricas**: A estrutura modular do *MetricCalculator* facilita a adição de novos algoritmos. Futuramente, propõe-se a inclusão de métricas mais complexas, como LCOM (*Lack of Cohesion of Methods*) e *Fan-in/Fan-out*, bem como a deteção automática de Code Smells (como por exemplo *God Class* ou *Long Method*).
- **Exportação e Integração**: Para facilitar a integração com outros sistemas, seria útil implementar a exportação para formatos estruturados como JSON ou XML, facilitando a integração dos resultados com outras ferramentas de visualização.
- **Interface e Visualização Avançada**: Evoluir a interface gráfica para incluir gráficos interativos diretamente na aplicação. Esta funcionalidade facilitaria a identificação imediata de anomalias (como classes com valores muito acima da média), tornando a deteção de problemas muito mais rápida e eficiente do que a simples leitura de dados numa tabela.

Em suma, o desenvolvimento deste projeto constituiu uma oportunidade valiosa para consolidar na prática os conceitos teóricos de Arquitetura e Engenharia de Software. A ferramenta final, juntamente com a documentação produzida, não só responde às necessidades imediatas de análise de código, como também demonstra a importância de um desenho arquitetural cuidado para garantir a longevidade e a qualidade dos sistemas de software.

Bibliografia

[1] GeeksforGeeks, "UML Class Diagram," 25 Agosto 2025. [Online]. Available: https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-class-diagrams/. [Acedido em Outubro 2025].

[2] H. Washizaki, Guide to the Software Engeneering Body of Knowledge (SWEBOK Guide), IEEE Computer Society, 2025.

[3] "Lines of Code metrics vs. the productivity metrics that matter," LinearB, 8 Maio 2025. [Online]. Available: https://linearb.io/blog/lines-of-code. [Acedido em novembro 2025].

[4] Lucidchart, "O que é BPMN?," [Online]. Available: https://www.lucidchart.com/pages/pt/o-que-e-bpmn. [Acedido em Outubro 2025].

[5] GeeksforGeeks, "Deployment Diagram in Unified Modeling Language(UML)," 23 Julho 2025. [Online]. Available: https://www.geeksforgeeks.org/system-design/deployment-diagram-unified-modeling-languageuml/. [Acedido em Outubro 2025].

[6] "Software Measurement and Metrics," geeksforgeeks, 2 Abril 2024. [Online]. Available: https://www.geeksforgeeks.org/software-engineering/software-measurement-and-metrics/. [Acedido em Dezembro 2025].

[7] T. Kuhn e O. Thomann, "Abstract Syntax Tree," 20 Novembro 2006. [Online]. Available: https://www.eclipse.org/articles/Article-JavaCodeManipulation_AST/#:~:text=The%20Abstract%20Syntax%20Tree%20is%20the%20base%20framework,plain%20Java%20source%20code%20in%20a%20tree%20form.. [Acedido em Outubro 2025].

[8] "Lines of Code metrics vs. the productivity metrics that matter," LinearB, 8 Maio 2025. [Online]. Available: https://linearb.io/blog/lines-of-code. [Acedido em Dezembro 2025].

[9] D. Spinellis e M. Jureczko , "Metric Descriptions," 22 Julho 2009. [Online]. Available: https://gromit.iiar.pwr.wroc.pl/p_inf/ckjm/metric.html.

[10] objectscript_Q, "Coupling Between Object classes (CBO)," [Online]. Available: https://objectscriptquality.com/docs/metrics/coupling-between-object-classes-cbo.

[11] Imagix, "Software Metrics," [Online]. Available: https://www.imagix.com/links/software-metrics.html. [Acedido em Novembro 2025].

[12] P. Pisakov, "Utility Tree in Software Architecture," 10 Junho 2023. [Online]. Available: https://www.pisakov.com/posts/utility-tree-in-software-architecture/. [Acedido em Novembro 2025].

[13] GeeksforGeeks, "Master-Slave Architecture," 23 Julho 2025. [Online]. Available: https://www.geeksforgeeks.org/system-design/master-slave-architecture/. [Acedido em Novembro 2025].

[14] GeeksforGeeks, "Layered Technology in Software Engineering," 23 Julho 2025. [Online]. Available: https://www.geeksforgeeks.org/software-engineering/layered-technology-in-software-engineering/. [Acedido em Novembro 2025].

[15] GeeksforGeeks, “MVC Design Pattern,” 3 Janeiro 2025. [Online]. Available: https://www.geeksforgeeks.org/system-design/mvc-design-pattern/. [Acedido em Novembro 2025].

[16] GeeksforGeeks, “Monolithic Architecture - System Design,” 7 Agosto 2025. [Online]. Available: https://www.geeksforgeeks.org/system-design/monolithic-architecture-system-design/ . [Acedido em Novembro 2025].